

THE ROSETTA CONJECTURE

— Domain-Driven Agentic Modernization —



The Rosetta Conjecture

Domain-Driven Agentic Modernization

Pierre Milet Llobet

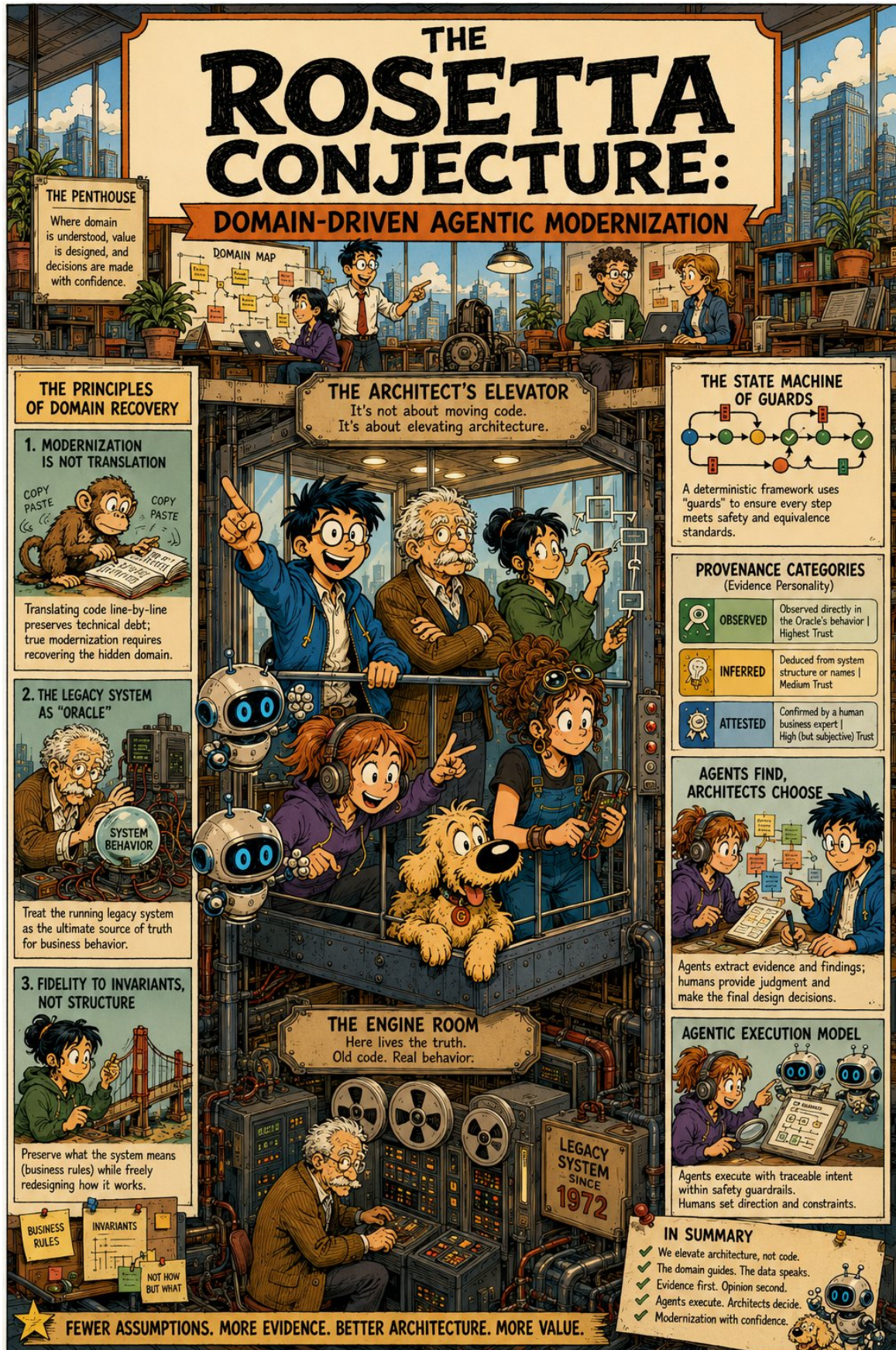
2026

Contents

About this book	3
Why Rosetta	3
Prologue. The modernization that does not modernize	8
I Why the domain and not the code	10
1 The graveyard has names	11
2 The relocation trap	13
3 The legacy as oracle	15
4 Fidelity to the invariant, not to the structure	17
II The agent on every floor	19
5 The architect's elevator, now with agents	20
6 Where the agent ends and the architect begins	22
7 The discipline of provenance	24
8 The harness	28
The deterministic substrate	28
The state machine	29
The guards are the book's gates	29
Signals and observability	29
Who governs the machine	30
III Explore before you productize	31
9 3X applied to modernization	33
10 Recovering the domain	35
11 Recover, then triage	37
12 The target context map	39
13 Tactical design of the core	41

IV Proving you did not break it	43
14 The transitive chain	44
15 Behaviour as executable specification	47
16 Black-box parity and the law of order	49
V Coexisting with two systems	51
17 The strangler, honest	52
18 Cutover without faith	54
19 Retiring the oracle without losing the reference	56
VI From Explore to Expand	58
20 This is a hypothesis	59
A Antipatterns	62
B Glossary	64

Modernizing is not translating code. It is recovering the domain.



About this book

Why Rosetta

For more than fourteen centuries no one could read the hieroglyphs. The scribes who carved them had died, and with them had gone the secret of how they sounded and what they said. The writing was still there, intact and mute. It was all in front of us, and none of it was understood.

The Rosetta Stone carried the same decree in three scripts, and one of them, the Greek, could still be read. That told us what it said. But knowing what it said was not the same as knowing how to read the dead script.

The key was another language, a living one. Egyptian had not vanished entirely: it had kept evolving into Coptic, still used in the liturgy of the Coptic Church, written in Greek letters and therefore readable. Champollion had no scribe to teach him. He had the living, phonetic form of the same language, and through it he recovered the sound and the sense of the dead script. The language that was still alive gave up the secret of the one that no longer was.

That is what this method does. The legacy system is the cryptic script, and those who wrote it are gone: no one documented the intent, the authors retired, the secret left with them. But the system has not died altogether. It still runs, still behaves, still speaks every day in production. That living behaviour is your Coptic: the readable form that outlives its authors and lets you recover the meaning they took away. You do not interrogate the scribes. They are gone. You interrogate the living language. I call it the oracle.

You do not translate the code into another language. You decipher the domain that the code stopped being able to explain.

And conjecture, because that is what I am offering. Not a method proven against hundreds of migrations, but a reasoned hypothesis about how it ought to be done when the domain is at the centre. A conjecture that asks to be walked, not believed.

This is not a book about tools. The tools it mentions, a compiler that rewrites an old language into a new one, a framework that organizes code into vertical slices, a graph that projects a system onto a navigable structure, an agent that reads, reasons and writes, will all age. Some will be obsolete before you finish reading. That is why they appear always as examples of an idea, never as the idea itself. What the book defends is a method, and a method is measured by the principles that hold it up, not by the parts that happen to embody it today.

The book has one centre and two axes. The centre is the domain. The two axes are borrowed. The first is Gregor Hohpe's architect's elevator, which runs from the penthouse, where strategy is decided, to the engine room, where the code executes. The second is Kent Beck's 3X model, Explore, Expand, Extract, which describes how an idea matures from a hunch into a standard. This book lives, avowedly, in the Explore phase. It does not sell you a method proven against hundreds of production migrations. It offers you an argument, coherent and reasoned, for why a method ought to work, and it invites you to walk it in fast cycles of learning to find out whether it deserves to move to Expand.

That honesty is deliberate. A book that overclaims is undone by a single failed engagement. A book that argues from principles endures, because principles are not refuted by anecdotes. If in five years today's tools are archaeology but the argument still stands, the book will have done its job.

One last note on the examples. There is a system to return to chapter after chapter, and it is called CardDemo. It is the mainframe-modernization sample that AWS publishes openly: thirty-one COBOL programs, with their copybooks, their batch and their CICS. It is not a toy invented for the occasion, it is real COBOL, with real coupling and at least one real

business quirk. And it is an example, not a bank's production system, which imposes a limit worth naming now: what works here suggests, it does not prove, that it will work over a larger and dirtier corpus.

CardDemo manages an issuer's credit-card accounts. Around the customer turn three objects: the account, which is the credit line with its limit and its balance; the card, tied to the account; and a cross-reference that reconciles the three. Each day it processes transactions against two rules it never lets cross: none exceeds the available credit of the cycle, none lands on an expired account. At the close of the cycle it computes interest by looking up the rate in a pricing table keyed by the account's group. Figure 1 shows its context map as recovery returns it: a core of account servicing and transaction posting, a hotspot where the account file is rewritten by four contexts at once, and clean seams to start from, such as security, isolated from credit. Keep one quirk in mind, because it will return: credit control looks at the exposure of the current cycle, not the balance the customer carries from earlier cycles. It may be intentional or it may be a defect. The business decides that, not the code, and it is the perfect example of something the method recovers faithfully but does not dare to correct on its own.

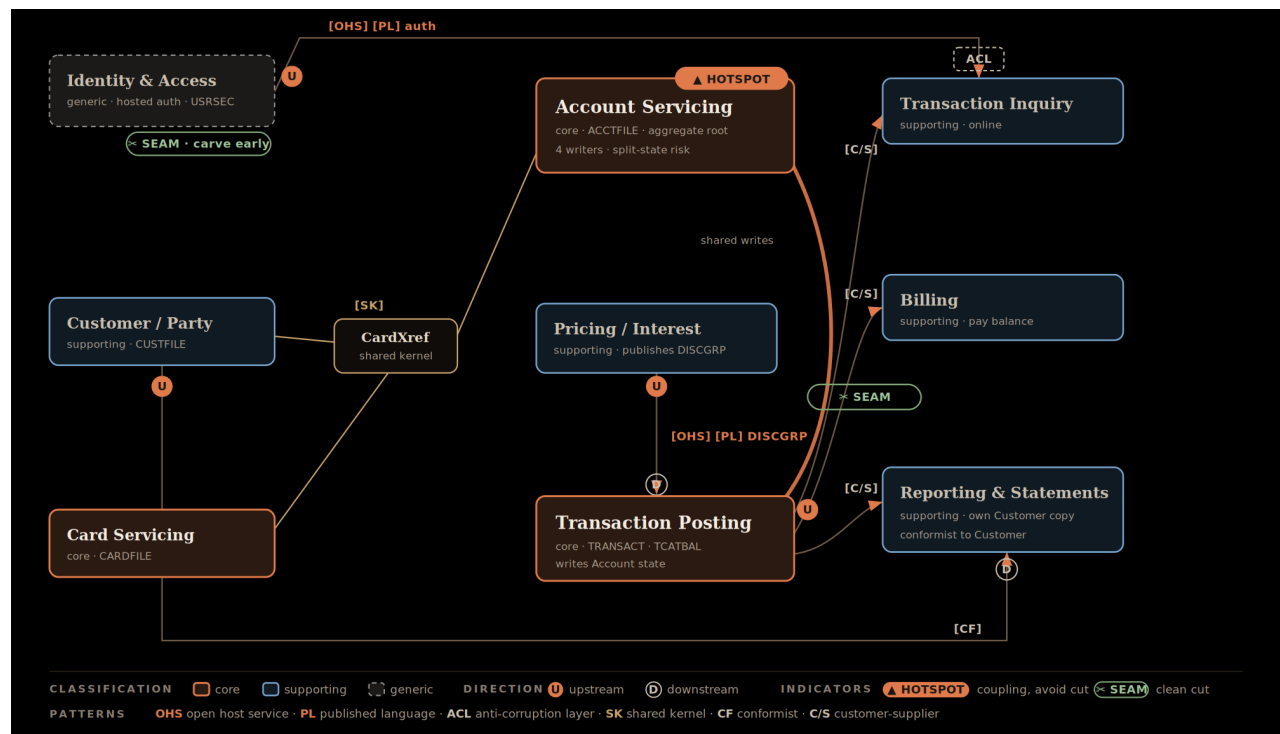


Figure 1: CardDemo context map. Core in orange, supporting in blue, generic in grey; the shared-write hotspot over the account file and the clean-cut seams.

CardDemo • step 1/11

CardDemo is the mainframe-modernization sample that AWS publishes openly: thirty-one COBOL programs, with their copybooks, their batch and their CICS. It is not a toy invented for the occasion, it is real COBOL, with real coupling and at least one real business quirk.

It manages an issuer's credit-card accounts. Around the customer turn three objects: the account, which is the credit line with its limit and its balance; the card, tied to the account; and a cross-reference that reconciles the three. Each day it processes transactions against two rules it never lets cross: none exceeds the available credit of the cycle, none lands on an expired account. At the close of the cycle it computes interest by looking up the rate in a pricing table keyed by the account's group.

The account record is declared by the copybook CVACTIONY. These six fields are enough to follow the thread:

```
01  ACCOUNT-RECORD .  
    05  ACCT-ID                PIC 9(11) .  
    05  ACCT-CURR-BAL          PIC S9(10)V99 .  
    05  ACCT-CREDIT-LIMIT      PIC S9(10)V99 .  
    05  ACCT-CURR-CYC-CREDIT   PIC S9(10)V99 .  
    05  ACCT-CURR-CYC-DEBIT    PIC S9(10)V99 .  
    05  ACCT-GROUP-ID          PIC X(10) .
```

Notice that two things coexist: the carried balance, ACCT-CURR-BAL, and the exposure of the current cycle, ACCT-CURR-CYC-CREDIT minus ACCT-CURR-CYC-DEBIT. That coexistence hides the quirk that will return: credit control looks at the cycle, not the carried balance. It may be intentional or a defect. The business decides that, not the code. And it is worth remembering: it is an example, not a bank's production system. What works here suggests, it does not prove, that it will work over a larger and dirtier corpus.

CardDemo entities around the customer

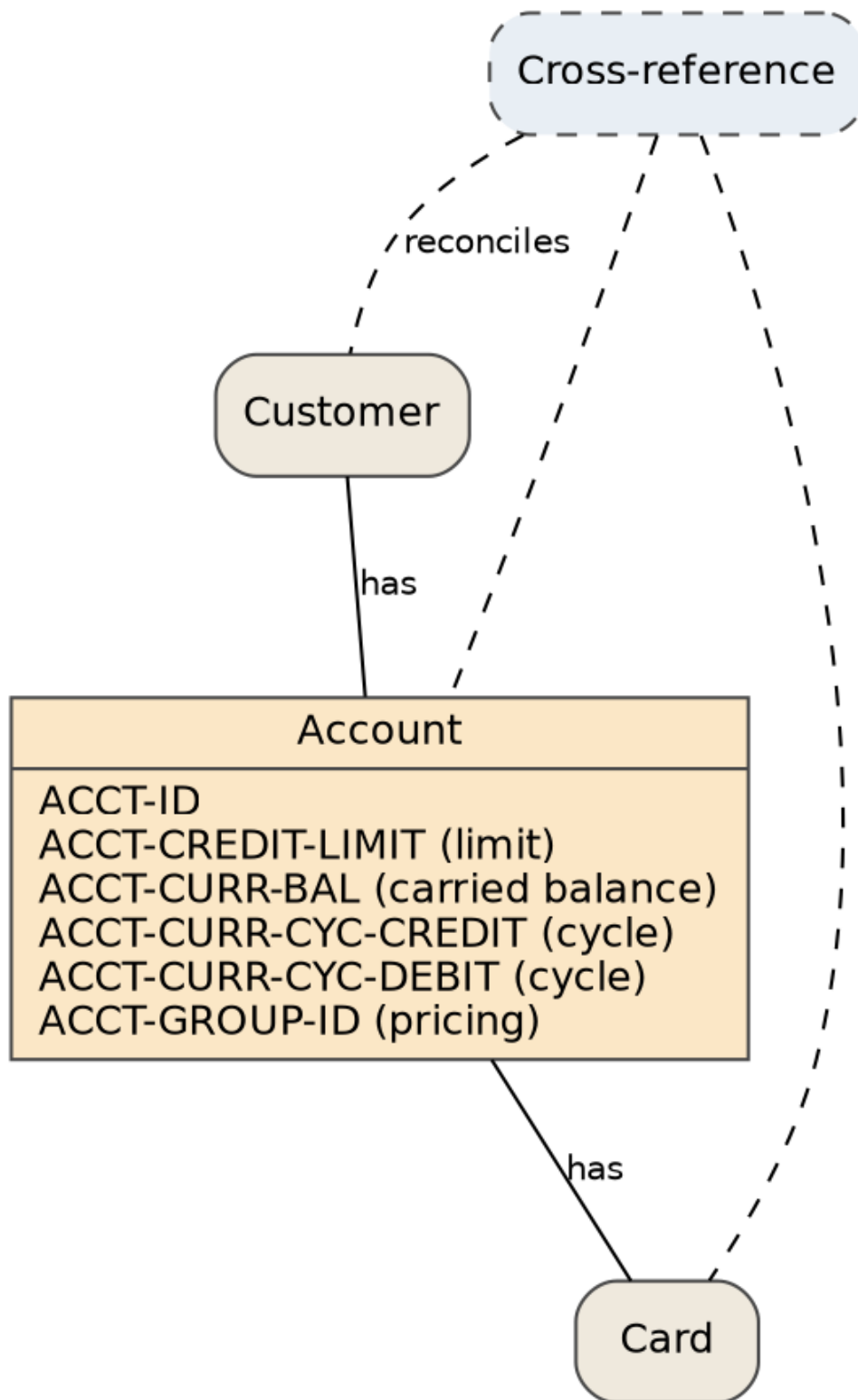


Figure. CardDemo's three entities around the customer: the account with its limit, its balance and its cycle; the card tied to the account; and the cross-reference that reconciles the three.

Prologue. The modernization that does not modernize

Start with what this book does not say.

It does not say “use AI to rewrite your COBOL”. It argues almost the opposite. The loudest pitch in the industry right now is that agents can translate millions of lines of legacy code at unprecedented speed. That pitch should worry you, because it is the same uncertified pipeline that has been failing for decades, only faster. Speed applied to a method that does not work does not produce modernization. It produces the next post-mortem, sooner.

AI’s first contribution to mainframe modernization is not generating code. It is making genuine understanding affordable, in systems where it had become impossible. The goal of this book’s method is not to rewrite. It is to recover the domain: to rebuild a true understanding of what these systems actually do, which business capabilities and rules they encode, where the risk concentrates, where the security exposure lives, and where the opportunity is. Strategic decisions about systems that run payments, policies and pensions tonight deserve to be taken with understanding, not with guesswork dressed up as a roadmap. And most of what you recover, you will rationally decide not to rewrite at all.

It is worth placing why this matters now. Decades of rehosting, recompilation and transpilation have already migrated the migratable: the peripheral systems, the manageable ones, those whose authors are still on the payroll. What remains submerged is everything those methods could not touch: transactional monoliths with decades of patches on top, authors long retired, documentation that lies. Half of the critical information in large organizations still lives there, and the pressure keeps growing: end of support, licence costs rising without return, regulation demanding evidence about black boxes, experts leaving without replacement. The submerged part was, until recently, the impossible. What has changed is that it is entering the territory of the possible. Not the easy. The possible.

Imagine you inherit a system. It has been in production for decades. It moves money, or lives, or cargo, something that matters. No one who wrote it is still at the company. The documentation, if it exists, lies. And yet the system works, day after day, with a reliability no recent project has matched. Your assignment is to modernize it.

The first temptation is the most reasonable and the most mistaken. It consists of treating the problem as a translation. You have code in an old language, you want code in a new one, so you translate. Today there are tools that do it almost on their own. A transpiler takes the source language and emits the target, line by line, structure by structure. The result compiles. It passes the tests, if there are any. And you have failed, though it will take you months to notice.

You have failed because you confused the symptom with the disease. The problem with a legacy system is not that it is written in an old language. The problem is that its design, the decisions about which concept lives where, which rule protects which datum, which part knows about which other part, was made decades ago, eroded with each patch, and was never looked at again as a whole. That design is the debt. Translating the language keeps the debt intact. You end up with the same tangle, now in modern syntax, and you have spent a budget moving it about. The syntax is new. The design is the same as ever. That is not modernization. It is relocation in a fair hand.

There is a more sophisticated variant that falls into the same trap. Instead of translating, you recompile. You take the conceptual binary of the old system and make it run on a new platform, without touching the source code. It is cheaper, faster, less risky in the short term. And it is still not modernization, because the design has not changed. You have changed the floor under the system’s feet, not the system. It is a legitimate relocation tactic, sometimes the right one, but calling it modernization is lying about what you have bought.

There is yet a third variant, the most sophisticated and the easiest to confuse with what this book defends. Instead of translating or recompiling, it decomposes. There are AI-assisted tools that split the monolith into business domains, grouping programs and files by their dependencies, and refactor each group into the new language. They speak of domains; some name DDD.¹ And they still fall short, because grouping the existing code into domains is not the same as recovering the domain model the code hides. A grouping tells you which programs touch one another. It does not tell you which invariant protects the account, nor which of its rules the business asked for and which is an accident from thirty years ago. And refactoring each group preserves its design, just as translation did, now with cleaner boundaries around the same debt.

This book starts from an uncomfortable claim. **Modernization is a domain problem, not a code problem.** The code is the way the domain was once expressed, under constraints that no longer exist. To truly modernize is to recover the domain the code hides, understand it, and express it again in an architecture that serves today's constraints. The target language is the easy part and the least important. The work is in the domain.

If you accept that, the rest of the book follows. If modernization is a domain problem, you need the conceptual tools of the domain, and the most mature one we have is Domain-Driven Design (DDD), the discipline of modelling software from the business domain and its language. You need to recover the domain model hidden in a system no one fully understands, which demands interrogating the system itself instead of trusting its code or its documentation. You need to redesign without breaking, which demands a discipline of verification that proves equivalence instead of assuming it. And you need to coexist with the old system while the new one is born, because no system that matters is switched off all at once.

There is a second thread in this book, and it enters now. As I write this, a new class of tool has begun to change the economics of all of the above. AI agents, programs that read, reason about what they read, and act, can traverse a legacy system at a speed and a scale that were previously impossible. But agents do not change the method. They change what the method can afford. A domain recovery that once cost months of a scarce expert can now cost days, which means you can afford to explore, get it wrong, and redo it. Tasks that used to belong to the Extract phase, mechanical and expensive, become Explore again, cheap and reversible. That is the real news. Not that agents write code, but that they make learning so cheap they change which phases of the work you can afford to walk.

That is why this is a book about agentic DDD applied to legacy modernization. The domain at the centre. The agents on every floor of the elevator, from the penthouse to the engine room, augmenting the architect at each level. And one question that runs through the whole text and gives it its subtitle. Where the agent ends and the architect begins. That boundary, you will see, is not a matter of convenience or of trust. It is an architectural decision, perhaps the most important one you will make.

Before theorizing why translation fails, it is worth looking at those who already tried. The graveyard has names.

¹The most visible example as I write this is AWS Transform for Mainframe, previewed as an Amazon Q Developer capability at re:Invent 2024 and generally available since May 2025. It decomposes the monolith into business domains from a dependency graph and refactors COBOL to Java, and it lists Domain-Driven Design as one of several decomposition strategies. The industry moves fast, so it is worth re-checking the state of any concrete tool before trusting this description.

Part I

Why the domain and not the code

Chapter 1

The graveyard has names

The prologue argued why translating fails. This chapter puts names to it, because the graveyard is real, it is recent, and its headstones can be read.

In April 2018, TSB migrated more than five million customers from the systems that Lloyds hosted for it to a new platform, over a single weekend, declared ready without the evidence to prove it. The data migration succeeded. The system did not. Nearly 1.9 of its 5.2 million customers were locked out of their accounts, some saw other customers' accounts, and the UK regulator ended up imposing a fine of £48.65m for governance and risk-management failures, not hardware ones. The lesson fits in one sentence. **“Data parity is not behavioural parity.”** Moving the data intact guarantees nothing about what the system does with it.

In 2010, Queensland Health's payroll system went into production with known defects and no way back. Around 78,000 employees were underpaid, overpaid, or not paid at all, for months. The cost of remediating it by hand was estimated at around 1.2 billion Australian dollars over years, and a commission of inquiry ranked it among the worst public-administration failures in the country. It was not an isolated calculation error. It was cutting over without having proven that the new system behaved as it should, and with no exit once it turned out it did not.

In 2023, a large European bank completed the migration of twelve million customers from its subsidiary onto its own platform. Migration “completed”. Then came months of locked accounts, rejected direct debits, and customers with no access to their funds, until the supervisor called the disruptions unacceptable and installed a special monitor inside the bank to force the remediation. “Completed” is not the same as “works”. Once again, data parity was not behavioural parity.

And then the verdict. In 2023, the Dutch state's independent IT audit body reviewed a factory-style conversion of COBOL to Java at the agency that pays the country's unemployment and disability benefits, built on state-of-the-art commercial transpilation.¹ The programme had stopped it, and the auditors concluded that stopping was right, because the approach was not producing maintainable software. Take the verdict seriously. Transpiling COBOL into Java-shaped COBOL adds almost nothing: you pay for a migration and receive the same system, harder to maintain, in a language that looks good on the slide but that neither the COBOL specialist nor the Java specialist can sustain. If understanding is missing, no change of syntax supplies it.

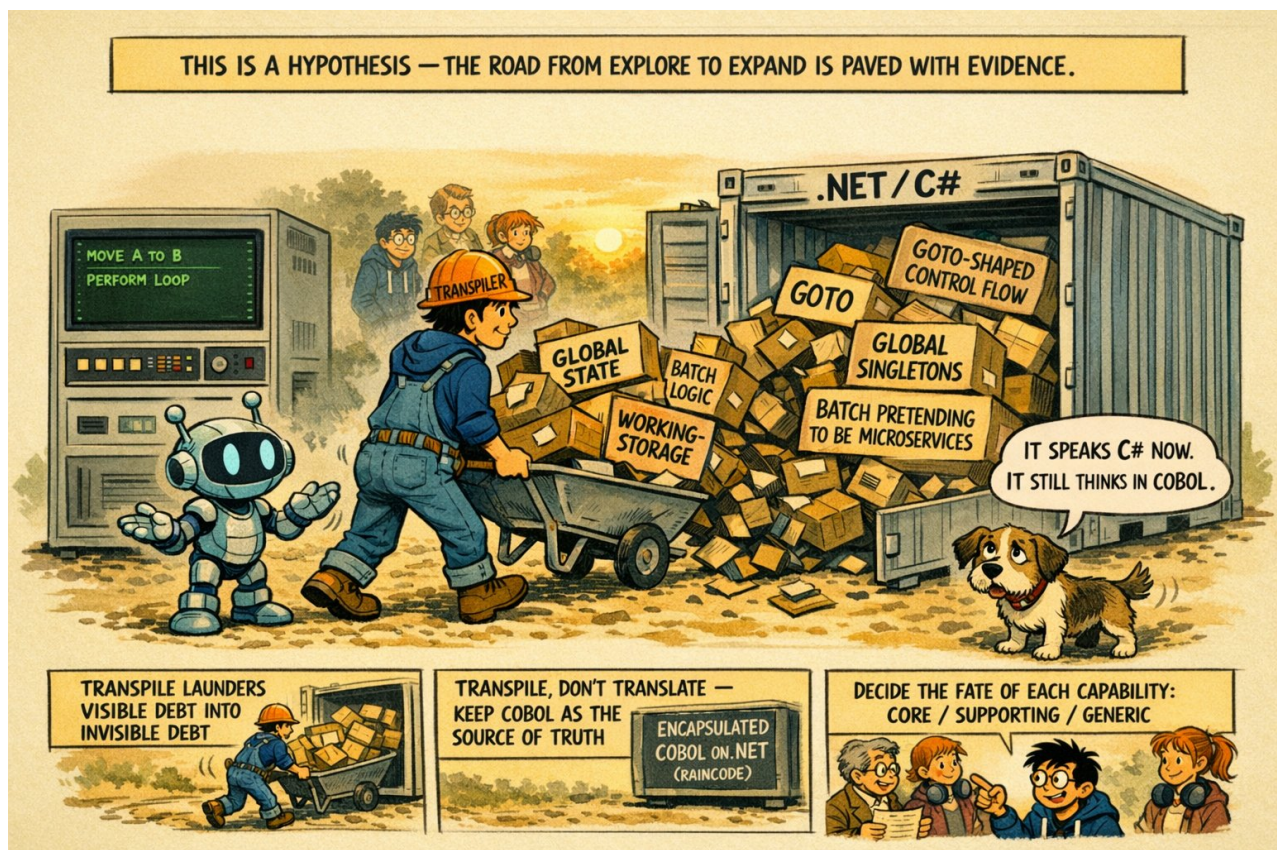
These four share neither a vendor nor a technology. They share a root, and it is not technical. It is governance. Someone asserted that the system was ready, or that it was equivalent, instead of proving it, and cut over on that assertion. Reality charged the difference. That is why the graveyard matters for this book: the uncertified pipeline the industry now wants to accelerate with agents is the same pipeline that filled these graves. And speed does not save a method that does not work; it only buries faster.

¹The Dutch UWV's OpenVMS programme (2020-2023) converted to Java the COBOL code of two benefits systems using a commercial automated-conversion tool, with Capgemini and subcontractor Blu Age. The Adviescollege ICT-toetsing (AcICT), the state's independent IT audit body, concluded in its opinion of 21 December 2023 that UWV had been right to stop the factory-style conversion, because it was not leading to maintainable software. Public sources for the four cases: the FCA and PRA final notices (TSB, 2022); the Chesterman commission of inquiry report (Queensland Health, 2013); BaFin's statements (Postbank/Deutsche Bank, 2023); and the cited AcICT opinion (UWV, 2023).

The way out of the graveyard is not a faster transpiler. It is to stop trusting assertions, documentation and memory, and to start interrogating the only witness that does not lie, the living system. But before that, it is worth understanding precisely why translation fails, because the failure is subtle and seductive. That is the next chapter.

Chapter 2

The relocation trap



Code is not the specification. It is the evidence.

It is worth looking closely at why translating fails, because the failure is subtle and seductive. A transpiler does exactly what it promises. It takes a structure from the source language and produces its equivalent in the target. A loop becomes a loop. A condition becomes a condition. A routine becomes a method. The mapping is faithful, and there lies the problem. Structural fidelity is precisely what you do not want to preserve.

Think about what is inside a typical legacy program. There is business logic, the rules that define what the system means, why an amount is rejected, when an operation is valid. And there is mechanism, the plumbing that makes that logic run on the platform of its day, how a file is opened, how a block of memory is passed from one program to another, how an input/output error is handled. In the old system, logic and mechanism are interwoven, because the platform of the day forced them to be. There was no other way.

When you translate line by line, you translate the two together. The old mechanism, which existed only to satisfy a platform you are abandoning, travels to the new system disguised as logic. You end up with modern code that opens files as if it were still on the old platform, that passes blocks of memory between modules as if memory were scarce, that handles errors with numeric codes that no longer mean anything. You have imported the plumbing of a house you have demolished. And because it is now in modern syntax, it is harder to spot, because it looks like legitimate new code.

The design suffers the same fate. If the old system crammed three distinct responsibilities into a single module because creating modules was expensive, the translation produces a new module with the three responsibilities. If two concepts that should have been separate shared a data structure to save space, they go on sharing it. The wrong boundaries are preserved with perfect fidelity. The coupling is kept. The absent cohesion stays absent. The only thing that has changed is the colour of the syntax.

This is where many well-meaning people make their stand. They say, fine, we translate first and refactor later. First a faithful equivalent, then we clean it up. It sounds prudent. It is a deeper trap, for two reasons.

The first is economic and psychological. Once you have a new system that compiles and works, the budget for the cleanup vanishes. The project is declared finished. The translated tangle stays, now blessed as a modern codebase, and the next ten years of maintenance will treat it as sacred. The refactor that was going to come later never comes. What was meant to be an intermediate step becomes the destination.

The second reason is more technical and more interesting. Refactoring demands knowing which behaviour you must preserve while you change the structure. If you have translated without understanding the domain, you do not have that knowledge. You do not know which of those three responsibilities of the module is business logic that must survive and which is mechanism that must die. You have no safety net to tell you whether your cleanup has broken something. Refactoring blind over code you do not understand is not modernization, it is roulette. We will return to this carefully, because the safety net, the proof that you have broken nothing, is one of the pillars of the method.

Recompilation, the other route, is honest in its way and therefore less dangerous, provided you do not lie about it. Making the old system run on a new platform without touching the code is a relocation. It has legitimate uses. It gets you off dying hardware, removes a vendor dependency, buys you time. What it does not do is modernize, because the domain is still expressed exactly the same, with the same wrong boundaries and the same interwoven mechanism. The mistake is not to recompile. The mistake is to believe you have finished.

There is a principle worth fixing right now, because it will return. **Changing the platform and changing the design are two distinct jumps, and they must never be made at once.** If you recompile to change platform, do it as a clean, isolated move, without touching the design. If you redesign, do it on a stable platform, without moving it under your feet. Whoever attempts both jumps in the same move is left with no foothold to diagnose what went wrong when something goes wrong, and something will. This forbidden double jump is one of the costliest and most common mistakes, and it has its own chapter further on.

So, if it is not translating or recompiling, what is modernizing. The short answer is recovering the domain and expressing it again. The long answer fills the rest of the book. But there is a prior turn, almost a change of attitude, without which none of the rest works. You have to stop seeing the old system as a burden to escape and begin to see it as the most reliable source of truth you have. That is the next chapter.

Chapter 3

The legacy as oracle

The legacy system has a bad reputation, and in part it has earned it. It is rigid, opaque, expensive to maintain, impossible to staff with people who know it. But there is one thing that system does better than any document, any expert and any specification you could write about it. It knows exactly what it does. It proves it every day, in production, without getting the rare cases wrong, the ones you do not even know exist.

That is the idea that changes everything. **The old system is not just a liability. It is an oracle.** An oracle, in the sense we give it here, is a source that answers a question about behaviour with the truth. You give it an input, it gives you the correct output, correct by definition, because it is what the system really does and what the business has accepted as correct for decades. You do not have to take anyone's word for how a surcharge is computed in the edge case, or what happens when two operations arrive in the improbable order. You ask the system and it tells you.

This reorders the priorities of recovery. Tradition says that to understand an old system you read its code and its documentation. The code is the better of the two sources, but it is treacherous, because it mixes logic and mechanism, because it is full of dead paths that never run any more, and because reading it tells you what the system appears to do, not what it does. The documentation is worse, because it describes what someone once wanted the system to do, and the system has spent years diverging from that intent without anyone updating the document. Both sources give you hypotheses. The oracle gives you facts.

The practical change is enormous. Instead of arguing for weeks about what an unintelligible routine means, you run it with chosen inputs and observe. Instead of trusting an expert to remember the special-case rule correctly, you generate the special case and look at the answer. The old system becomes a laboratory. You run experiments on it. The questions that used to produce meetings now produce observations.

For the oracle to be truly useful it needs two things, and here is where the tools come in, as examples of a concept. The first is that it answers fast. An oracle that takes a night to give you an answer is useless for exploring, because exploration needs short cycles, ask, observe, adjust the question, ask again, dozens of times a day. That is why it pays to have a copy of the old system that runs locally, in milliseconds, not on the production mainframe with its job queue. One way to get this is to recompile the old system to a modern platform you can run on your own machine, in a container, as many times as you like. The compiler that does that recompilation is an example of a tool. The durable concept is the fast, local oracle, a faithful replica of the old behaviour that answers in the time of a thought cycle.

Notice something elegant. Recompilation, which in the previous chapter was a false modernization if you mistook it for the destination, here is exactly the right tool for a different purpose. You are not recompiling to deliver a system. You are recompiling to build yourself an oracle. The same technique, despised as a goal, is valuable as an instrument. The difference is in what you use it for.

The second thing the oracle needs is fidelity. It has to behave like the original, not resemble it. If your replica diverges from the real system in the rare cases, it will be lying to you precisely where you will most rely on it. That is why the replica's fidelity is not assumed, it is validated, by comparing it against the real system over enough cases. An unverified oracle is a rumour with a good reputation.

There is an honest caveat worth not hiding. The oracle only knows what it is asked. It knows the behaviour over the

inputs you give it, not over the ones you do not think of. If a path in the old system never runs in your experiments or in the real traffic you observe, its behaviour goes uncaptured, and it is behaviour that may matter on the least expected day. The oracle does not eliminate uncertainty, it bounds it and makes it visible. This will have consequences when we talk about which inputs to choose to interrogate it, because choosing the questions well is the difference between an oracle that covers the boundaries where the bugs live and one that only confirms what already happens every day.

For now, keep the change of attitude. The system you were asked to replace is, while it lives, your best teacher. Before you retire it you have to learn from it everything you will ever need, because when it is switched off, that source of truth disappears. Much of the method consists of squeezing the oracle with discipline while it is still switched on.

Chapter 4

Fidelity to the invariant, not to the structure

If the old system is the oracle and what you are after is to preserve its behaviour, a tricky question arises. Preserve what, exactly. Because if the answer were to preserve everything, there would be no modernization possible, you would be bound to keep every last detail of the old system, including the tangle you wanted to remove. Fidelity misunderstood returns you to the translation trap.

The right answer demands a distinction that is the conceptual heart of this chapter and one of the keys to the whole method. There are two very different things an old system contains, and only one of them is sacred.

The first is the what. The business rules, the invariants, the rules that must always hold, the semantics of each operation. An amount can never exceed a certain limit. An operation on an expired entity is rejected. A certain calculation rounds in a certain way and the result matters to the last decimal because it is money. This is the problem space. It exists independently of how the old system chose to implement it. It would survive even if you rewrote the system from scratch in any language. This is what you must preserve with absolute fidelity.

The second is the how. The concrete structure the old system chose to satisfy those rules. Which module held which logic, which data structure was shared by which concepts, in what order things were done, where the boundaries were drawn. That the account file is rewritten by four distinct programs, as happens in CardDemo, is mechanism of the old solution: the business never asked for four writers, it only asked that the balance add up. This is solution space, and it is an old solution, made under constraints that no longer exist. It is not sacred. It is, in fact, precisely what you have come to change.

Here is the sentence worth carving. **Fidelity is to the invariants and the semantics, not to the structure.** You preserve what the system means. You are free to redesign how it achieves it. That freedom is no minor permission, it is the entire prize of modernization. Without it you would be condemned to copy the old design. With it you can redraw boundaries, separate what was mixed, give each concept its place, and still guarantee that the system goes on meaning the same thing.

An image helps. No map of the world is faithful to everything at once. The Mercator projection is faithful to bearings, the line of constant bearing comes out straight, and that is why it serves navigation; in exchange it distorts area, and Greenland looks as large as Africa while being fourteen times smaller. An equal-area projection is faithful to surface and distorts shapes. It is not a failing of the cartographers. It is impossible: a sphere cannot be flattened without breaking something, and Gauss proved it. Every flat map chooses what to be faithful to and lets the rest distort.

To modernize is to make that choice. You choose to be faithful to the semantics, to what the system means, and you let the structure change shape. The old system was a map faithful to an old machine, as Mercator is to navigation. The new system will be another map of the same domain, faithful to something else. The world, the business, is the same. What changes is what you are faithful to as you draw it.

This distinction has a consequence that seems surprising and that will later be a technical pillar. If what you preserve is the semantics and not the structure, then two systems can be equivalent in behaviour and completely different inside. The new system can have other boundaries, other modules, another order, and still give the same answer to the same question. That means the proof that you have broken nothing cannot consist of comparing the innards of the two systems, because by design they are different. It has to consist of comparing what is observed from outside, the behaviour, not the

implementation. That is the seed of black-box parity, which is what we will call verifying equivalence by looking only at observable inputs and outputs, never the inside. We will develop it in full in Part IV.

The distinction also lights up where the most dangerous mistakes of a modernization hide. There are pieces of an old system that look like mechanism but carry a business decision inside. The classic example is the handling of a technical error. When the old system does not find a datum and reacts in a certain way, is that reaction pure plumbing, detecting an absence, or is it a business rule in disguise, deciding that an absence is equivalent to a certain default value with real consequences? If you treat it as mechanism and discard it, you have silently lost a rule. If you treat it as a rule and keep it, you drag plumbing along. Telling which is which cannot be done by reading alone, because the code interweaves the two. It is done by asking the oracle what business consequence that behaviour has. The silent loss of semantics, keeping the technical shell and losing the rule it hid, is one of the costliest antipatterns, and it appears in its appendix.

It is worth closing with an image that sums up the three ideas of this first part. Think of the recovered domain as a model you will be able to dress in many different architectures. The model, the rules and the semantics, is one and is stable. The architectures possible over that model are many. Translation fails because it copies an old architecture without recovering the model. The oracle lets you recover the model with facts instead of rumours. And fidelity well understood tells you what of the model is untouchable, the semantics, and what is free, the structure. With that in hand, the question stops being how do I translate this code and becomes which of the many architectures possible over this domain best serves today's constraints. That is an architect's question. And architects, it turns out, now work accompanied by agents.

CardDemo • step 2/11

In CardDemo, the account file is rewritten by four different programs, two in batch and two online:

```
*   CBTRN02C   transaction posting   (batch)
*   CBACT04C   interest calculation  (batch)
*   COACTUPC   account add and edit  (CICS)
*   COBIL00C   bill payment          (CICS)
      REWRITE FD-ACCTFILE-REC FROM ACCOUNT-RECORD
```

The business never asked for four writers; it only asked that the balance add up. The four writers are mechanism of the old solution, not domain semantics. That spread of the write is the hotspot that will return when we recover the map.

Part II

The agent on every floor

Chapter 5

The architect's elevator, now with agents

Gregor Hohpe proposed a metaphor that serves to understand what an architect does, and which here we will stretch a little beyond its original purpose. Imagine an organization as a building. In the penthouse strategy is decided, where the business is going, what matters. In the engine room, many floors below, the code executes and things really happen. In between there are intermediate floors, the architecture, the teams, the processes. Most people live on a single floor. Those in the penthouse talk of vision and do not know what is in the binaries. Those in the engine room know every bit and do not know why it matters. The value of the architect, says Hohpe, is that they ride the elevator. They go up and down. They translate strategy into structure and structure into strategy. They connect floors that would otherwise not speak to each other.

In a modernization, the elevator is literal. In the penthouse are the domain's questions, what concepts this business has, what rules govern it, where the natural boundaries between its parts lie. In the engine room is the old system, its code, its data structures, its interwoven routines. And the work of modernizing is precisely an elevator journey, going down to the engine room to recover what the system does, going up to the penthouse to understand what that means in domain terms, stopping on the intermediate floor to decide the architecture, and going down again to verify. Whoever can only read code cannot modernize, because they do not see the domain. Whoever knows only the domain cannot either, because they cannot recover anything from a system they cannot read. Modernization is an elevator trade.

Here enters the book's second thread, and it is worth saying precisely so it does not age. A class of tool has appeared, agents, that can ride that elevator. An agent, in the sense that matters here, is a system able to read a great quantity of material, reason about it, and produce results, code, models, analysis, proposals. It is not magic and it is not a colleague. It is a tool with a new and important property. It operates on every floor. It can read code in the engine room. It can propose domain boundaries in the penthouse. It can generate the scaffolding of an architecture on the intermediate floors. Before, each floor demanded a different tool and a different person. Now there is a class of tool that traverses them all.

This is what justifies the adjective agentic in the title, and it is worth understanding well why. It is not that agents are a one-off accelerator of a concrete task, like a faster compiler. It is that agents are present at every level of the work, augmenting the architect on every floor of the elevator at once. The architect who used to go down alone to the engine room now goes down with a tool that reads ten times faster and does not tire. The one who went up to the penthouse to propose a map of bounded contexts now goes up with a tool that has already read the whole system and brings candidates. DDD remains at the centre, intact, because DDD is the discipline of putting the domain first, and that was not invented by agents nor will it be replaced by them. What changes is that now the domain is recovered, modelled and verified with a companion on every floor.

Two opposite mistakes about this must be resisted. The first is disdain, believing that agents are autocomplete with pretensions and that the method changes in nothing. The second is fascination, believing that agents do the work and that the architect is superfluous. Both mistakes fail in the same place, they do not understand what changes exactly. What changes is not the principles, it is the economics. And when the economics of an activity change enough, what activities you can afford changes, and that does alter the method, not in its principles but in its scope.

The clearest example is given by Beck's own 3X model, which we will see in detail further on. Beck distinguishes three phases in the life of an idea. Explore, when you are looking for what works and almost everything you try is discarded.

Expand, when something works and it has to be scaled. Extract, when it is already stable and you squeeze its efficiency. In a traditional modernization, recovering the domain was treated as if it were Extract: a final act, expensive, slow, done by scarce experts, that you did once and accepted as good even if it was bad, because redoing it was unthinkable. With an agent that recovers in days what used to cost months, that same task drops in cost until it becomes Explore. You can afford to recover the domain, look at it, not be convinced, throw it away and recover it again another way. What was an irreversible commitment becomes a cheap experiment. That is the deep transformation. Not that the agent recovers the domain, but that it makes it so cheap that recovering it stops being a single, solemn act and becomes a fast learning cycle.

So the agent rides the elevator with the architect, on every floor, and makes the journey so cheap that it changes which journeys you can afford. But there is a floor where the agent does not decide, it only proposes. Knowing where that boundary is, and why it has to exist, is the next chapter, and it is the most important decision in the book.

Chapter 6

Where the agent ends and the architect begins

If the agent traverses every floor, the natural and dangerous question arises. Then what does the architect do that the agent does not. The lazy answer is what the agent cannot do yet, and it is lazy because it ages with every new version of the agent and because it treats the boundary as a temporary technical limitation. The right answer is different and does not age. There is a class of decisions the architect must make not because the agent cannot, but because they must not be delegated, and understanding why is understanding the method.

Let us start with how an agent reasons, because the key is there. An agent does not reason well over raw source code. The code is long, repetitive, full of irrelevant detail, and it interweaves logic with mechanism. What an agent does well is reason over projections of the system, representations that distil one dimension of the system and discard the noise. A graph that shows what calls what. A model that shows what datum flows to what decision. A structure that shows what rules protect what entity. Over those projections the agent reasons with power, because the projection has already done the work of separating signal from noise. The code is the territory, and the projection is the map. Further on we will step back and see that the code itself is also a map, the one its authors drew of a deeper territory, the business; for now this level is enough. And agents, like explorers, work better with maps.

This already draws a first boundary. Building good projections, deciding which dimension of the system deserves a map, is an architect's work. The agent navigates the map, but which map to draw for which question is a design decision. A wrong map leads a very powerful agent to a wrong conclusion with total confidence, which is the worst class of error.

But the deeper boundary has to do with the difference between finding and choosing. When you recover a domain from an old system, there are things you discover and things you decide. You discover that a certain rule exists, because it is there, in the oracle's behaviour, and you can point to it. That is a finding. It has a source. An agent can do it, because it consists of observing and reporting what is observed. By contrast, when you decide that two concepts the old system mixed should go in separate contexts, you did not find that anywhere. The old system did not tell you. You chose it, in the light of your judgement about what best serves today's business. That is a design decision. It has no source in the old system, because its source is you.

The boundary between the agent and the architect is the boundary between finding and choosing. The agent finds. It reports what the system does, with its source, because finding is observing. The architect chooses. They decide which architecture, which boundaries, which rules to consolidate or separate, because choosing is an act of judgement that does not derive from any observation. And the reason this boundary must not be crossed is not that the agent is incapable of proposing an architecture, of course it can propose one. The reason is that a proposal without a source, presented as if it were a finding, contaminates the model with decisions disguised as facts. And a model where you cannot tell what you recovered from the system from what you invented yourself is a model you cannot trust for anything important, because you do not know how much of it is truth about the business and how much is a tool's opinion.

There is an old contrast that gives this boundary its name. Aristotle put knowledge in the observable, in what the world is seen to do. Plato put it in the ideas, in what things ought to be, which is not read anywhere because it is not out there,

it is reached by thinking. The oracle is Aristotelian: it says what the system does, and it says it because it is observed. The architect, when they choose, is Platonic: they decide what the system ought to be, and that is found neither in the code nor in the oracle, not because they have not looked well, but because it is not there. To find is to observe what is. To choose is to reason what ought to be. That is why a choice can never carry the label of a finding. It does not come from the same place.

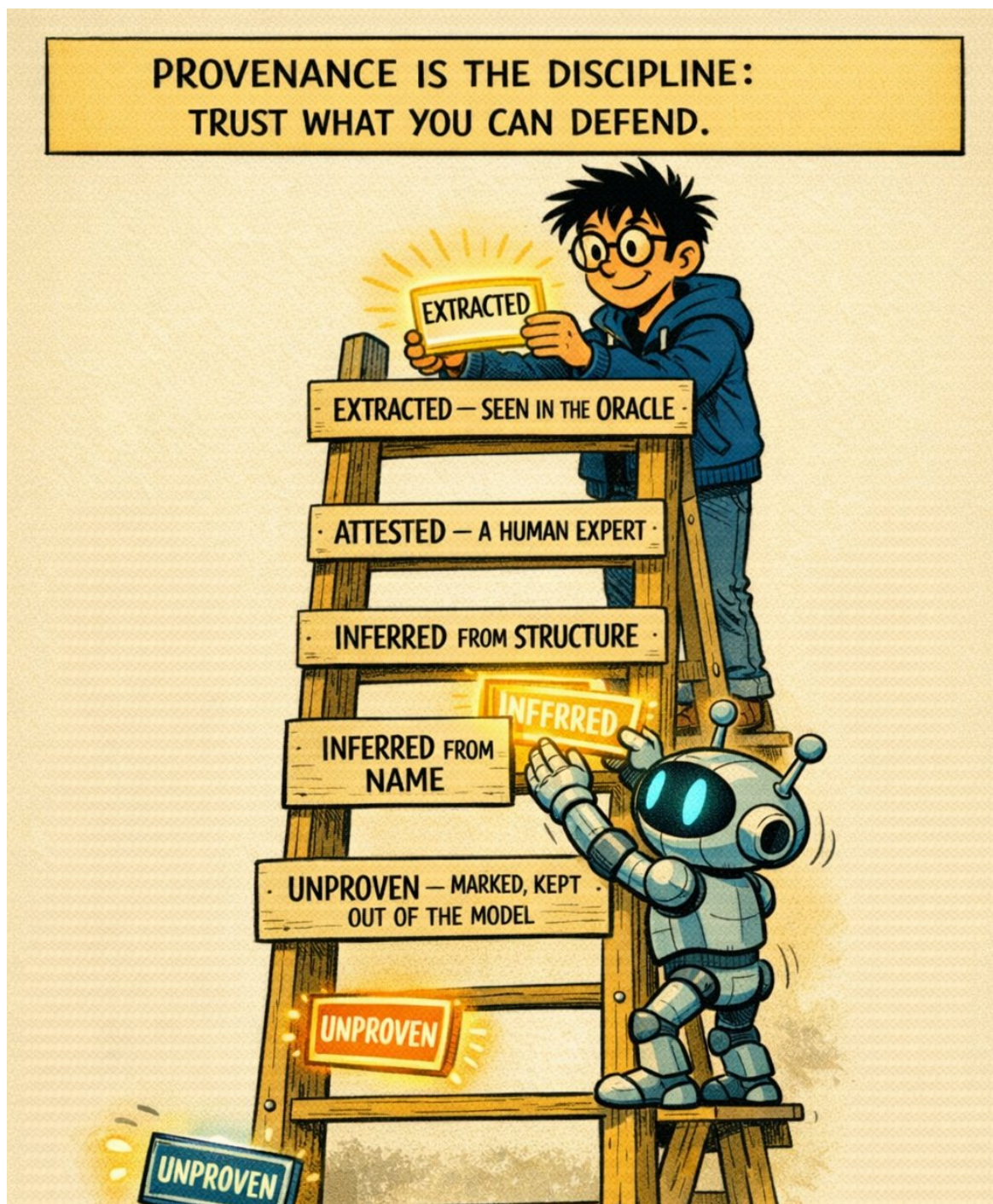
This turns the boundary into an architectural decision, not one of convenience. You do not say the agent goes this far because beyond it gets things wrong. You say the agent finds and the architect chooses because mixing the two destroys the reliability of the model. It is an epistemic boundary, about what kind of knowledge each thing is, and it turns out to coincide exactly with the boundary of who does what. The recovered, which has a source, the agent can bring. The decided, which has no source other than judgement, the architect must put, and it must be marked as such.

It is worth insisting on an asymmetry that people underrate. The agent can propose design options, and it should, because there it is very useful. The right thing is for the agent, after recovering the domain, to present the architect with a spread of possible architectures, each with its evidence and its consequences, especially its consequences for the later verification. This option is cheaper to test but consolidates two rules into one. This other is more faithful but more expensive to verify. The agent is excellent at laying that table of options. What it cannot do is sit at the table and choose for you, because choosing is committing judgement, and judgement answers to the business, not to a tool. The agent serves the menu. The architect chooses.

A practical question remains. If the recovered and the decided have to be told apart, how are they told apart in practice, inside the model, so that anyone reading it knows what they are looking at. The answer is a simple and rigorous discipline, and it is the next chapter.

Chapter 7

The discipline of provenance



A domain model recovered from an old system is a mixture of things of very different origins. Some you extracted directly from the observed behaviour. Others you inferred from the structure, they look true but you have not confirmed them. Others a business expert who has been with the system for thirty years confirmed for you. Others are design decisions you made yourself. And others are simply conjectures you have not yet proven. They all coexist in the same model, and from outside they look the same, sentences that describe the domain. That appearance of sameness is a danger, because it invites you to trust them all equally, when they deserve very different degrees of trust.

The solution is a discipline I ask you to take seriously even if it sounds bureaucratic, because it is what holds up the reliability of the whole method. **No statement enters the model without declaring its provenance.** Every rule, every boundary, every invariant you write carries a mark that says where it comes from and, therefore, how much you can trust it.

The categories of provenance are few and clear. There are extracted statements, observed directly in the oracle's behaviour, the most reliable, because you have seen them happen. There are statements inferred from structure, deduced from how the system is organized, plausible but unconfirmed, because the structure can deceive. There are statements inferred from a name, suggested by what something is called, the weakest, because names in old systems lie often. There are statements attested by an expert, which a human who knows the business confirms. And there are unproven statements, conjectures marked honestly as such so that no one takes them for facts. The concrete marks you use do not matter, what matters are the distinctions they capture. CardDemo gives the perfect example. Credit control looks at the exposure of the current cycle and not the balance the customer carries from before. That behaviour is extracted, you saw it happen in the oracle. But whether it is a deliberate rule or a decades-old defect is an unproven statement, and that part the method does not decide: it takes it to the decision register for the business to resolve.

There are a couple of subtleties worth pausing on, because it is where the discipline proves its worth. The first. What is attested by an expert is not the same as what is extracted from the oracle, even though both seem firm. The expert can misremember, can describe how they believe the system works instead of how it works, can be describing a version from ten years ago. Their testimony is valuable, but it is testimony, not observation. That is why it carries its own mark, distinct from the extracted. When the expert's testimony and the oracle's observation disagree, the discipline of provenance warns you that you have a conflict to resolve, instead of letting you choose blind. And almost always the oracle wins, because the system does what it does, whatever the memory says.

It is also worth distinguishing two experts the word hides. The business expert inhabits the problem space: they know why the rule exists and what would happen if it changed. The mainframe veteran, the one who has maintained the system for twenty years, inhabits the solution space: they know which twist in the code is a platform idiom and which hides a decision, where the cases that bite are buried, and whether the replica acting as oracle behaves like the original where it matters most. Their testimony is the attested channel for disambiguating mechanism from logic, exactly the boundary where chapter four placed the most expensive mistakes. Agents do not displace them: they promote them. They stop spending their years on trivial maintenance and move to doing what only they can do, interpreting the oracle.

The second subtlety connects this chapter with the previous one and closes the idea of the boundary. There is a golden rule. **What can cite no source is not a finding, it is a choice, and it must leave the model of the recovered for a decision register.** If you write a statement about the domain and, on trying to mark its provenance, you discover you cannot point to where it comes from, that you did not extract it or infer it or have anyone attest it, then you are not describing the old system. You are deciding something about the new one. That statement does not belong to the model of the recovered. It belongs to the register of design decisions, which is a different matter altogether, the input to the redesign, not part of the portrait of the domain.

This rule is exactly the boundary of the previous chapter, finding versus choosing, made operational. Provenance is the mechanism by which the boundary between the agent and the architect stops being an intention and becomes a practice you can audit. Anyone can take the model and ask, of each statement, where it comes from. If everything recovered has a source and everything without a source is in the decision register, the boundary is clean. If you find statements without a source camouflaged among the recovered ones, you know that someone, a human or an agent, has slipped in a choice disguised as a finding, and you know the model is contaminated right there.

Some will say this is too much ceremony for an engineering job. The answer is that the cost of the ceremony is small and the cost of not having it is enormous and deferred. Without a discipline of provenance, six months later no one remembers whether the edge-case rule was observed in the old system or invented by someone in a meeting, and that difference is the difference between a business fact and an opinion that may be wrong. Provenance is cheap to maintain

if you maintain it from the start, and ruinously expensive to reconstruct if you did not. It is, at bottom, the same lesson as the verification we will see further on. Better to prove than to assume, and better to know where each thing comes from than to trust that someone remembers.

With the boundary drawn and provenance in place, we have the two halves of the agentic method. But a boundary that must be remembered and a discipline that must be willingly followed are fragile. What holds them up when no one is watching is not goodwill, it is a machine. And that machine is the subject of the chapter that closes this part.

CardDemo • step 3/11

Credit control lives in CBTN02C, paragraph 1500-VALIDATE-TRAN. It looks at the cycle exposure, not ACCT-CURR-BAL:

```
COMPUTE WS-TEMP-BAL = ACCT-CURR-CYC-CREDIT
                  - ACCT-CURR-CYC-DEBIT
                  + DALYTRAN-AMT
IF ACCT-CREDIT-LIMIT >= WS-TEMP-BAL
  CONTINUE
ELSE
  MOVE 'OVERLIMIT TRANSACTION'
  TO WS-VALIDATION-FAIL-REASON-DESC
END-IF
```

That behaviour is extracted: you saw it happen in the oracle. Whether it is a deliberate rule or a decades-old defect is, by contrast, an unproven statement. The method does not decide it: it takes it to the decision register for the business to resolve.

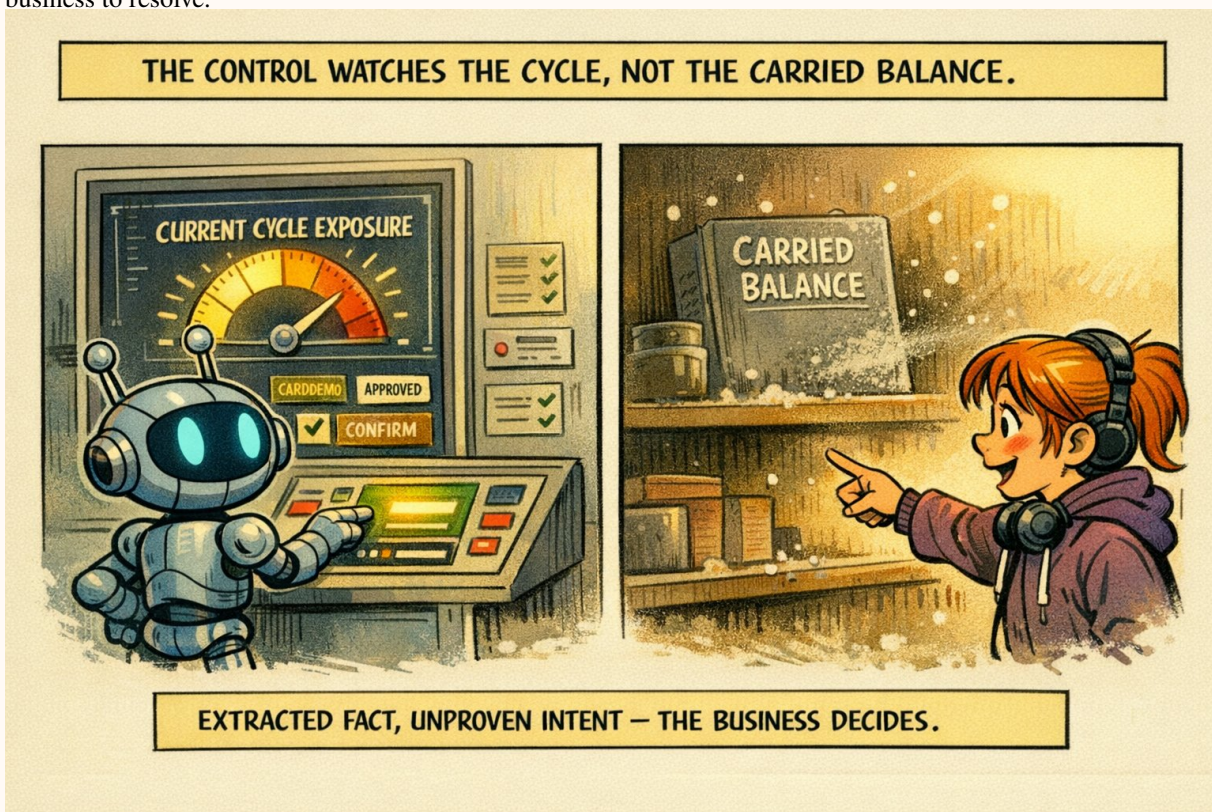


Figure. The credit quirk over a timeline of two billing cycles: the control looks at the exposure of the current cycle and ignores the balance carried from the previous one.

Chapter 8

The harness

The two previous chapters drew a line and asked for a discipline. The line separates what the agent finds from what the architect chooses. The discipline demands that every statement declare its provenance. Both are correct, and both are fragile, because they depend on someone remembering to respect them. A line that exists only in the head of whoever drew it is crossed the first day the calendar gets tight. A discipline held up only by goodwill is abandoned in the first rush.

The question of this chapter is what makes the line and the discipline hold when no one is watching. **The answer is that it is not will that holds them, it is a machine.** I call that machine the harness. It is not a concrete tool. It is a way of organizing the agent's work so that the boundary and provenance stop being intentions and become properties of the system, checkable and hard to skip.

The deterministic substrate

In chapter six it was said that the agent does not reason well over raw code, but over projections, maps that distil one dimension of the system. A question was left unanswered, and it is the most important of all. If the agent reasons over the projection, who builds the projection, and why trust it.

There is a tempting and wrong answer: let the agent build it, reading the system and drawing the map. It is wrong because then the projection would inherit the agent's fallibility. An agent that draws the map it is then going to interpret can mis-trace an edge, invent a call that does not exist, omit one that does, and then reason with total confidence over a map that lies. The costliest error is not the agent that gets a map wrong as it reads it. It is the agent that gets it wrong as it draws it and no one notices.

That is why the projection is not invented by the agent. **The projection is derived mechanically from the system, by a deterministic process that does not opine.** The code and the data are analysed with fixed rules, and the result is poured into a navigable structure: what calls what, what datum flows to what decision, what condition protects what state. That pouring is reproducible. If you run it twice over the same system, you get the same map. And each element of the map can point to which part of the system it came from, which is the provenance of the previous chapter applied to the substrate: each edge cites its origin in the code.

Over that deterministic substrate, the agent does what it does well. It searches, relates, proposes. A search by meaning finds candidates: this structure looks like an aggregate, this condition looks like an invariant. And the deterministic graph verifies whether those candidates hold, whether the call really exists, whether the datum flows as believed. The search proposes, the graph disposes. The agent never verifies against its own memory, which invents with aplomb. It verifies against the substrate, which does not invent.

In practice. The substrate usually materializes as a property graph queried alongside a semantic index. That pairing, a semantic search that proposes and a graph that verifies, is what the industry calls GraphRAG. The state machine can be a lightweight library of states and transitions. The agents of each state are coordinated by an orchestration framework. The concrete names will change. What does not change is the idea: deterministic substrate, states with bounded access, guards that cannot be skipped.

The state machine

The substrate makes what the agent reads reliable. What is missing is making what the agent does reliable. Recovering a domain is not a single act, it is a sequence: read, propose, confront with the oracle, ratify, decide architecture, verify. If that sequence is left to the judgement of the moment, it falls into disorder. Ratification happens before confronting. Architecture is decided over a half-recovered model. What is not yet decided gets verified.

The solution is not a new idea. It is the oldest way computing has of being reliable: a state machine. The Turing machine, the abstract model with which the theory of computation begins, is exactly that, a machine whose next step is determined by its state and its input, with no room for surprise. And it is, point by point, the opposite of an agent, which, given the same input, can answer different things. The harness lives on that contrast: it puts the unpredictable, the agent, to work inside the predictable, the machine.

The harness organizes the sequence as that state machine. **Each state has a single goal, the agent that pursues it, the context it needs and nothing more, and a bounded set of tools it can reach.** One state to recover the tactics. Another to confront candidates with the oracle. Another to ratify. Another to propose architecture. The agent that recovers cannot reach the tools that decide, because in that state it does not decide. The one that verifies cannot touch the model, because in that state it does not model. The context of each state is cleared on entry, so that the noise of one task does not contaminate the next.

This is not bureaucracy. It is what stops the tasks from mixing. An agent with access to everything and memory of everything tends to do everything at once, and doing everything at once is exactly how a choice disguises itself as a finding and a conjecture slips in as a fact. The state machine is the boundary of chapter six made topology: the agent cannot cross from finding to choosing because they are different states, with different access, and the step between them is guarded.

The guards are the book's gates

Between state and state there is a guard. A guard is a deterministic condition that must hold for the machine to advance. It is not a piece of advice or a list someone reviews. It is a lock. If the condition does not hold, the transition does not happen, and there is no way to push it through by hand without leaving a trace.

Here the harness gathers pieces that in other chapters appear loose. **What the book calls gates are guards of this machine, not checklists of good intentions at the end of a branch.** Ratification is a guard: only what can defend its provenance passes into the model, and the doubtful does not transition, it stays or is confronted again. The law of order, which we will see in Part IV, is a guard: you do not enter the quality-refactor state until the equivalence guard is green. The cutover, in Part V, is a guard: you do not cut until its conditions are met, equivalence, coverage, shadow and resolved divergences.

Four disciplines that seemed independent turn out to be the same thing seen in different places: transition conditions of a machine that does not let you advance until they are met. That is the difference between a method that asks you to do things in order and a method in which disorder is, quite simply, impossible.

Signals and observability

A guard is only as good as the information it decides with. If the equivalence guard asks “do old and new match” and the answer is a yes or a no without nuance, it is useless. It needs to know what they match on, with how much coverage, which cases were tested and which were not. That information is the signals: what the system emits about itself while it works.

Observability is not an operational luxury added at the end. It is what lets the guards decide with grounds and the boundary be auditable. **Every decision the machine makes leaves a trace with its provenance, so that anyone can ask afterwards why it advanced, and the machine answers.** Why this rule was ratified. With what evidence this equivalence was closed. What coverage the specification had when the cut was allowed.

Coverage deserves a word, because it is the one that puts a number on the honest residue. In the oracle chapter it was said that the old system only knows what it is asked, and that what is not exercised goes uncaptured. Coverage is the

measure of how much has been exercised. It does not eliminate the residue, no process captures what never manifested, but it bounds it and makes it visible. A guard that opens with poor coverage does not certify equivalence, it certifies that they matched where you looked. Observability is what stops you from confusing the two.

Who governs the machine

It remains to say where the human is. The temptation, when the machine works, is to put them inside the loop, approving each step, or to take them out entirely and let it run on its own. Both are mistakes.

The architect is not inside the loop, they are on the loop. They do not approve each transition, that is done by the deterministic guards, which are there for that and do it better than a tired person on a Friday. The architect watches the machine, reads its traces, and decides at the points where the machine, by design, does not decide: where the boundary is, which architecture, what to do with what has no provenance. **The agent operates inside the machine. The architect governs the machine. They are not the same job and they are not delegated to each other.**

This closes the argument of Part II. The agent finds, and the machine guarantees that only what has a source enters as a finding. The architect chooses, and the machine guarantees that the choice is marked as a choice and is not disguised as something else. The boundary is no longer an intention that must be remembered. It is the shape of the machine.

With the boundary drawn, provenance in place and the harness enforcing them, the two halves of the agentic method fit together: the agent that finds and the architect that chooses, separated by a machine that does not let them be confused. We have the how. One question of attitude is missing, the one that decides when it is worth using this seriously and when not, and why a young method treated as mature fails. There Beck enters.

Part III

Explore before you productize

THE DOMAIN ONTOLOGY

— THE LANGUAGE YOU VERIFY IN —

1 The first move in modernisation is not choosing a target architecture. It is recovering the domain, stripped of the mechanics four decades have wrapped around it.

2 What you recover is the ubiquitous language of Domain-Driven Design: the single vocabulary business and engineering both speak, with no translation layer in between.

3 It does two jobs.

1. UNDERSTANDING & DESIGN
It gives the people who never read a line of COBOL a shared, precise understanding of what the business does, and a way to design the future in business terms instead of file formats.

2. VERIFICATION
And then it does the job almost no one asks of it. It becomes the language you verify in.

4 Watch the legacy run, in production or in the oracle (recompiled to run as a behavioural reference you can replay), and you capture behaviour. Inputs in, outputs out.

5 Translate that behaviour into a domain rule a business person would recognise and sign off, and both systems can be held to it.

6 A transaction would push an account past its credit limit, and the legacy declines it.

7 Agents earn their place in two roles.

8 VERIFICATION: PROPOSE

THE ONTOLOGY IS A SUBSTRATE IN ITS OWN RIGHT.
One ontology can be realised by an infinity of target solutions. The architecture is free to change. The meaning is not. Fidelity is owed to the invariant, not to the structure.

THE ONTOLOGY (THE DOMAIN)

ACCOUNT CUSTOMER TRANSACTION
CREDIT LIMIT PAYMENT DECLINE ...

COBOL MAINFRAME COMPILED COBOL MODULAR MONOLITH MICROSERVICES EVENT-DRIVEN SYSTEM AGENTIC SYSTEM

LEGACY (TODAY)
Account: A123
Limit: \$1,000
Txn: \$1,200
DECLINED

THE RULE (DOMAIN)
A TRANSACTION THAT BREACHES THE CREDIT LIMIT IS DECLINED.

NEW SYSTEM (TOMORROW)
Account: A123
Limit: \$1,000
Txn: \$1,200
DECLINED

DISCOVERY: PROPOSE
An agent reads what no human reads end to end and proposes the domain rules buried in the mechanics.

CANDIDATE RULE
A TRANSACTION THAT BREACHES THE CREDIT LIMIT IS DECLINED.

RATIFIED

EXECUTABLE SPECIFICATIONS (BDD)
Runs against Legacy (Oracle) PASS
Runs against New System PASS

ONE ONTOLOGY. INFINITE IMPLEMENTATIONS. ARCHITECTURE IS FREE TO CHANGE. MEANING IS NOT.

COBOL MAINFRAME COMPILED COBOL MODULAR MONOLITH MICROSERVICES EVENT-DRIVEN SYSTEM AGENTIC SYSTEM

Chapter 9

3X applied to modernization

Kent Beck observed that software lives three very different phases, and that confusing them is one of the most common causes of disaster. He called them Explore, Expand, Extract, the three X's. In Explore you look for what works, and the uncomfortable truth of this phase is that most of what you try does not work, so the value is in trying cheaply, learning fast and discarding without regret. In Expand something has worked and suddenly you have to grow to serve a demand that spikes, and the problem stops being what to do and becomes holding up. In Extract you already know what you do and you do it at stable scale, so the game is to squeeze efficiency, polish, optimize. Each phase asks for a different mindset, and applying the mindset of one to another is the mistake. Optimizing in Explore is polishing something you are going to throw away. Exploring in Extract is introducing chaos where stability was needed.

The thesis of this chapter, and one of the most important attitude decisions in the book, is that **a young modernization methodology is in Explore, and treating it as if it were in Extract is the fastest road to failure**. This applies at two levels, and it is worth separating them because it is easy to mix them.

The first level is the method itself. This book describes a method that is in its Explore phase. It has not been validated against hundreds of production migrations. It is a reasoned hypothesis, an argument for why it ought to work. Treating it as a mature process, codifying it into a rigid corporate methodology and rolling it out at scale before having walked it in the small, would be committing Beck's mistake in its purest form, productizing something you are still exploring. The right attitude is the opposite. Walking the method in small, fast cycles, over bounded pieces, learning from each cycle, discarding what does not work, until the evidence justifies moving it to Expand. The method earns the right to scale, it does not arrogate it.

The second level is each concrete modernization engagement, be it a bounded project or a multi-year enterprise transformation. It also starts in Explore, even if organizational pressure pushes you to pretend it starts in Extract. The pressure says, we already know what has to be done, let us plan the eighteen phases and execute. It is a lie, and it is an expensive lie. At the start of an engagement you do not know enough, and it is worth admitting why. You are not an expert in the client's business. You are an expert in modernizing, which is another trade. You do not know the real domain hidden in the system, you do not know where the natural boundaries are, you do not know which parts are logic and which mechanism, you do not know where the rare cases that will bite you are. That is why the domain is not brought from the head of whoever modernizes. It is recovered from two sources: from the system, which shows what it does, and from the business experts, who need not be engineers and who put the name and the why to what the system only shows itself doing. Faking a detailed plan over that ignorance produces a plan of fiction that reality destroys in the first month.

The alternative is to start with what deserves to be called an architecture spike, borrowing the term from agile practice. A spike is a time-boxed experiment whose goal is not to produce a deliverable but to produce knowledge. You take a slice of the system, thin but complete, ideally one that touches the most feared parts. Thin matters. It is not a horizontal layer, it is not only recovering or only verifying. It is a needle that pierces every floor of the elevator at once, from the engine room to the penthouse: recovery, redesign and verification over a narrow strip of the domain. It crosses everything, and a little of each thing. Not to deliver it, but to learn. At the end of the spike you know things no prior plan could give you. You know how much it really costs to recover the domain of this system. You know whether your oracle is faithful.

You know where the rare cases are. You know which boundaries make sense. With that knowledge, the plan you write afterwards is a real plan, not a fiction.

This is where the agentic cheapening of chapter five changes the rules in a deep way. In a world without agents, spikes are expensive, because recovering the domain by hand is slow, so the temptation to skip them and plan blind is strong, and often it wins. In a world with agents, the spike is cheap, because the recovery that used to cost weeks costs days. And when the spike is cheap, you can afford several, explore several slices, several boundary hypotheses, several candidate architectures, before committing to any. What used to be a luxury almost no one allowed themselves, exploring for real before deciding, becomes the default option. The agent does not only speed up the work. It makes prudence affordable.

There is a discipline that goes with 3X and that is worth stating plainly. Do not productize prematurely. The temptation, when a spike goes well, is to declare victory, freeze the approach and apply it en masse. It is the temptation to jump from Explore to Extract without passing through Expand, to turn a successful experiment into a standard before having tested it on several different cases. A single successful spike does not prove the method works, it proves it worked once, which is a great deal less. The signal that you are ready for Expand is not that something worked, it is that you understand why it worked well enough to predict when it will not. Until you have that, you are still in Explore, and it is fine to be in Explore. It is where you learn.

This attitude, walking in fast cycles before committing, fits with the elevator of chapter five and with the original agile philosophy, the real one, the one from before the word filled up with ceremonies: short cycles, learning from the real world, discarding without regret what does not work. Each cycle is a complete elevator journey, you go down to recover, up to model the domain, you stop on the intermediate floor to decide the architecture, you go down to verify. The difference between doing it well and badly is not in the floors, which are the same, but in the speed and the humility of the journey. Fast, to learn a lot per unit of time. Humble, to throw away without regret what the journey reveals does not serve. With that attitude in place, the first real journey is to go down to the engine room to recover the domain. That is the next chapter.

Chapter 10

Recovering the domain

Recovering the domain is the central act of modernization. It is the foundation from which everything else hangs, and a crooked foundation is not straightened higher up. The goal is to produce a faithful portrait of the domain the old system embodies, expressed in the terms of Domain-Driven Design, stripped of mechanism, and with each statement marked by its provenance. Let us call that portrait the recovered model. It is not the old system and it is not the new one. It is the domain, the what, separated from the old how and not yet committed to any new how.

It is worth being very precise about what this model is and is not, because confusion here poisons everything else. The recovered model describes the current state of the domain, not the desired one. It is a portrait of the as-is, not of the to-be. This surprises whoever expects modernization to start by drawing the ideal system. No. It starts by understanding with precision what is there, because you cannot preserve the semantics of something you have not understood, and preserving the semantics is the fidelity contract of chapter four. The ideal system comes later, and it comes as a design decision, marked as such, not as part of the portrait.

The recovered model has two layers, the two classics of DDD. The tactical layer describes the inside of each part, the entities and their identities, the aggregates and the invariants they protect, the value objects, the relationships. The strategic layer describes the whole, the context map, the boundaries between the different parts of the domain and how they relate. Recovering both is the work. The tactical tells you what rules live inside each concept. The strategic tells you where the seams of the domain are, which is where later you will be able to cut.

Here is a nuance many people miss and that is worth nailing, because it looks like a contradiction and is not. If what you recover is the domain, the what, why does the recovered model also contain structure, which is of the how. Because a faithful portrait cannot ignore how the system is organized today. The recovered model is a new map of the business, and on that map you note two distinct things, with distinct status.

The first is the mandatory layer: the invariants and the semantics, the problem space. It is the what, it survives any redesign, and it is sacred. The second is an informative layer: the structure as the system runs and is observed today, the boundaries it has in place, which module keeps what, where the seams are. It is solution space, but not of an ideal design nor of one someone imagined decades ago, but of the living system as the oracle shows it. You recover it because it is valuable information about how the system understands its own domain today, not because it is sacred. It is, in fact, precisely what you have come to change.

That is why there is no contradiction. You recover the domain, and along the way you note the current structure as data, marked as re-decidable. Confusing the two layers is the mistake. If you treat the recovered structure as sacred, you go back to the translation trap. If you treat the invariants as negotiable, you break the system. The map draws everything, but it tells you of each stroke which layer it belongs to.

The recovery process, in the agentic method, splits the work according to the boundary of chapter six. The agent finds. It traverses the system, or rather, it traverses the projections of the system, the maps that distil its dimensions, and extracts candidates, this structure looks like an aggregate, this condition looks like an invariant, this datum looks like the identity of this entity. Each candidate comes with its provenance, I extracted it from behaviour, I inferred it from structure, the name suggests it. The architect ratifies. They look at the candidates, confront them with the oracle when there is doubt,

confirm or reject them, and above all decide what to do with what has no clear source.

There is an idea about ratification worth highlighting because it corrects a common misunderstanding. Ratification is not where uncertainty is resolved. It is where uncertainty is forbidden from entering. It sounds the same but it is not. Resolving uncertainty sounds as if in the ratification meeting the doubts are discussed and an agreement is reached about the doubtful. Wrong. The doubtful is not ratified, it is marked as doubtful and sent to be verified against the oracle or investigated. What is ratified is what is already solid, what has a firm source. Ratification is a filter that lets through only what can defend its provenance, not a tribunal that turns conjectures into facts by consensus. If you leave ratification having blessed as certain something that entered as doubtful, without new evidence, you have corrupted the model with the authority of a signature.

An honest warning to close. Recovery is never complete. There will always be behaviour on paths that are not exercised, cases the oracle has not seen because it was not asked, dormant rules that wake only in rare circumstances. That is irreducible residue, and the right attitude is not to pretend it does not exist but to mark it. The honest recovered model includes its own gaps, the zones where it knows it does not know. Those gaps are not a failure of recovery, they are its most valuable part, because they tell you exactly where the new system may surprise you, and therefore where to watch. A model that pretends to be complete lies. One that marks its gaps tells you where to look.

With the domain recovered, portrayed, marked and with its gaps flagged, the elevator goes up. But before deciding any architecture, you have to decide what even deserves one. That is triage, and it is the next chapter.

CardDemo • step 4/11

The agent does not read the thirty-one programs raw. It reasons over a projection: a graph where each program and each file is a node, and each access is an edge. Over that projection, the hotspot is not something you search for, it draws itself. A query brings it to light:

```
MATCH (p:Program)-[:WRITES_FILE]->(f:File {name: 'ACCTFILE'})
RETURN p.name AS program
ORDER BY program
```

The answer is four programs writing the same file. That was in no documentation. It was in the behaviour, and the graph made it visible. It is the first concrete proof that the agent reasons better over the map than over the territory.

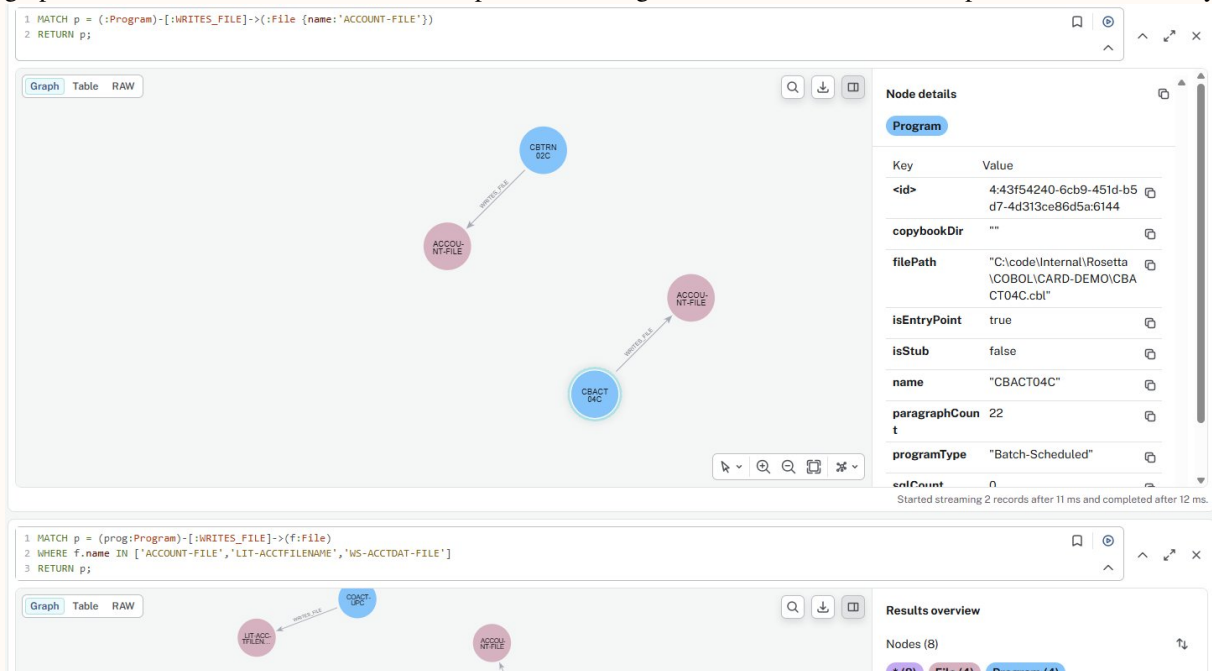


Figure. The Atlas graph in Neo4j: the `WRITES_FILE` query over the account file, with the programs that write it represented as nodes of the graph.

Chapter 11

Recover, then triage

Once the domain is recovered, when you can finally see what you own, the strategy stops being one-size-fits-all and becomes a triage, in plain Domain-Driven Design terms. **The recovered domain is not a rewrite list. It is the map with which you decide what to rewrite, what to encapsulate and what to retire.** The map of Figure 1 already carried the answer drawn: the colour is the triage.

Generic subdomains, the business's commodities, go to packaged solutions. There is no glory in maintaining bespoke code for problems the market solved years ago, and every commodity you retire frees scarce people for the work that does matter.

Supporting subdomains, and any code that is stable and simple, get encapsulated and left alone. These workloads stop consuming your senior COBOL experts, who are needed where only they will do, attesting, interpreting the hotspots, validating the fidelity of the replica. With the recovered domain knowledge and the oracle in place, an agent or a junior developer can make minor, well-guarded changes safely: adding a field to a copybook should not require a hero. And what makes that work safe is not faith in the agent: it is the specification captured at the service's contract, the regression net of Part IV applied to the encapsulated.

Here is a position you will rarely hear in an AI article: keep the COBOL. COBOL that is stable, correct and quietly doing its job is an asset, not an emergency. The mature move is to encapsulate it behind an explicit contract, an open host service with a published language, so the rest of the estate consumes capabilities instead of internals. If the platform itself must move, relocate the COBOL as COBOL, recompiled onto a modern runtime and containerized, behind that same contract. Better still, when the economics allow it, keep it where it is and expose it the same way, through a remote endpoint over HTTP. Either way the principle is identical: **wrap to invoke, isolate so as not to inherit. Encapsulation buys optionality; transpilation spends it.**

The comparison with transpilation deserves to be made explicit, because both get sold as exits from the mainframe and they resemble each other in nothing. Recompiling keeps the artefact that gets maintained: the COBOL remains COBOL, the team that knows it remains competent, and the compiler guarantees by construction the semantics of the data, the packed decimal, the encoding, the collation, the precision, which is exactly where a conversion breaks in silence. Transpiling replaces the artefact with generated code nobody wrote and nobody wants to maintain, the graveyard's verdict. That is why the same technique that built a trustworthy oracle in chapter three is here a worthy destination: it preserves the semantics by design instead of promising them.

In practice. A compiler such as Raincode recompiles the COBOL to .NET IL and runs it on the modern runtime, preserving by design the representation of the data, COMP-3, EBCDIC, collation and decimal precision, so that the program running in a container remains, for maintenance purposes, the same COBOL. The compiler is the instance; what endures is the criterion: for what you will not redesign, choose the path that keeps the artefact and the semantics, not the one that replaces them.

This does not contradict the relocation trap from the start of the book; it completes it. There the mistake was confusing relocating with modernizing the core, selling a platform change as if it were a transformation of the domain. Here, for the code you have deliberately decided not to redesign, relocating and encapsulating is exactly right. The double-jump

law still holds: relocation is a clean, isolated move, and you do not pretend it modernized anything. The difference is in being honest about what you bought.

Rewriting is reserved for the code that deserves it: the core of the business, where change is constant, churn is high, complexity is real, and your senior developers already spend most of their time. There, and only there, redesign pays: re-architecting the core buys agility, cuts technical debt and restores maintainability. And that rewritten core must be designed from day one to coexist with the remaining legacy, with DDD's strategic patterns and the legacy displacement patterns, because for years your estate will be a federation of old and new, not a replacement.

Hence the inversion that runs through the whole book, seen from the portfolio. Most of what you recover, you will rationally decide not to rewrite. Recovery is not the obligatory prelude to rewriting; it is what lets you not rewrite where it does not belong, and direct every euro of redesign to the one place where it pays off. For the core you do decide to rewrite, now it is time to decide the architecture. And there, at last, the architect chooses. Their first choice is not about code, it is about the map, and it is the next chapter. From here on, the rest of the book speaks mostly of that core; the encapsulated estate does not leave the story, it takes part in the map through its contract and reappears every time the new has to coexist with it.

CardDemo • step 5/11

Lifted from graph to context map, this is what recovery returns of CardDemo. It is not a design, it is a portrait: the structure as the living system runs it today. The core, account servicing, transaction posting and card servicing, in orange; the supporting, where customer, interest, inquiry, billing and reporting sit, in blue; identity and access, the one generic, in grey. And in red the hotspot, the account file rewritten by four contexts at once. The colour is already the triage: orange is rewritten, blue is encapsulated, grey is bought.

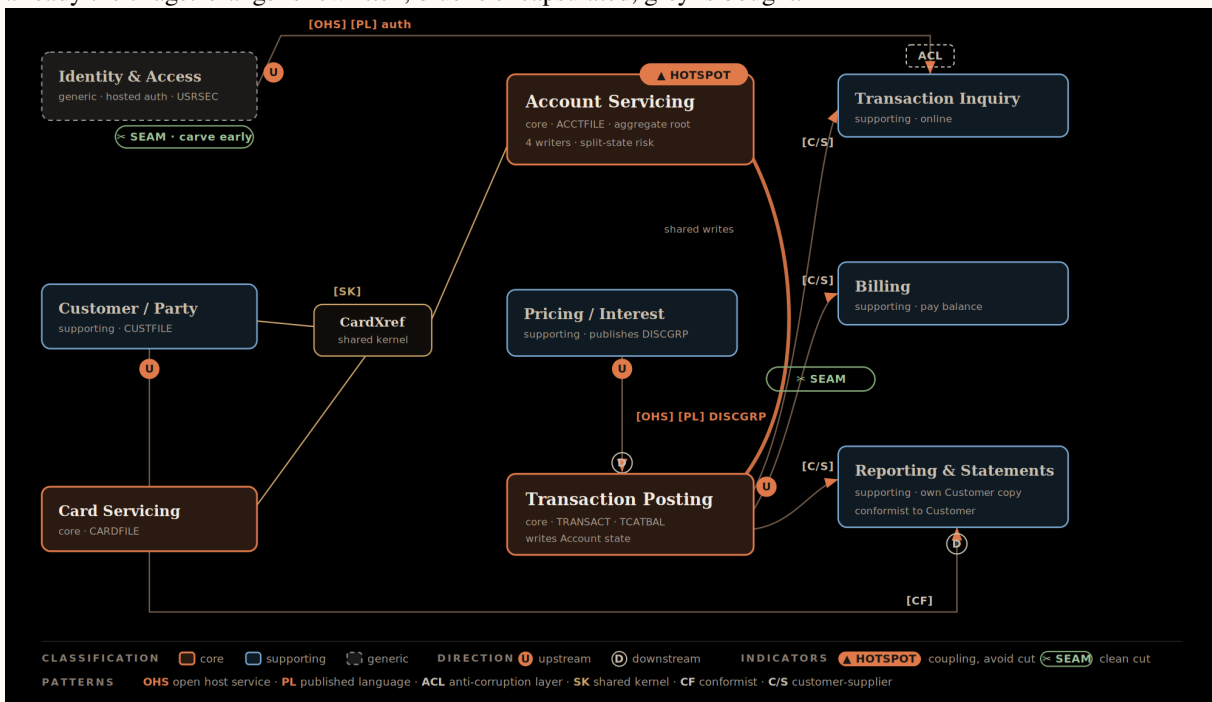


Figure. CardDemo's as-is context map as recovery returns it: core in orange, supporting in blue, generic in grey, and the shared-write hotspot over ACCTFILE in red.

Chapter 12

The target context map

The first act of redesign does not happen in code. It happens on the map. Before writing a line of the new system, the architect decides the target context map: which bounded contexts the resulting estate will have, which model and which language live inside each one, and how they relate to one another. **The first architecture choice is not about code, it is about the map.** It is the act translation skips entirely, and it is where the modernization is won or lost as design.

Remember that you arrive here with two maps in hand. The recovered model brings the as-is map, the structure as the living system shows it, noted as re-decidable data. The target map is something else: it is a decision. Over one recovered domain many possible maps fit, and choosing one is the architect's judgement, not the agent's finding. Every boundary the target map keeps from the as-is is a decision to keep. Every boundary it corrects is a decision to correct. Both get marked as decisions, with the usual provenance discipline, because the target map is not recovered from anywhere: it is chosen.

It is worth being precise about what is decided when a context is drawn, because it is more than a box with a name. A bounded context is the boundary within which a model of the domain and its language are coherent. And the ubiquitous language, let it be said without ambiguity, is not one for the whole business: it is one per context. In CardDemo, "account" does not mean exactly the same to the servicing that governs it as to the reporting that aggregates it, and that difference is not a defect to unify, it is information to protect with a boundary. Forcing a single language over different meanings produces the worst of models, the one that looks coherent and is not. The map exists so that each meaning has its home.

The map includes all contexts, not only the ones you are going to rewrite. Here the triage of the previous chapter becomes cartography. The encapsulated COBOL is not residue off the chart: it is a bounded context in full standing, with the model it has, its contract and its relationships drawn. A map that portrays only the new is propaganda. The honest map portrays the entire federation of old and new you will operate for years, because the relationships that demand the most care are exactly the ones that cross that seam.

The relationships between contexts have names, and using them disciplines the design. The context that offers capabilities to many consumers publishes an open host service with a published language. The context that consumes and wants to protect its model raises an anticorruption layer. They are not alternatives to choose between: they are the two ends of the same relationship, the open host and the published language live on the side that offers, the anticorruption layer on the side that consumes, and a healthy relationship can have both. Between contexts whose teams negotiate priorities there is a customer and supplier relationship. Where you accept the other's model as is, without translating, you are a conformist, which is cheap and honest when the foreign model is good. And where integrating costs more than it is worth, the right relationship is separate ways: duplicating a little and not integrating is also a relationship, the cheapest of all when it applies.

CardDemo shows the target map in miniature. The core, account servicing and transaction posting, gets rewritten, and the account-file hotspot is resolved on the map before it is resolved in code: the account state comes to have a single context that owns its writes, and the others consume it by contract. Security, stable and isolated, stays as an encapsulated context behind an open host. Reporting consumes downstream, conformist if the published language serves it, with its own anticorruption layer if it needs its own model. Three kinds of relationship, one domain, and none of those arrows was dictated: all of them are choices the map leaves written down.

The map also has a price the agent can put in front of each option. Every boundary it redraws relative to the as-is diverges from the oracle and makes the later verification more expensive; every boundary it keeps makes it cheaper. An ambitious map pays for its ambition in Part IV. That is not a reason to avoid ambition, it is a reason to be ambitious with the price in sight, and to remember that the map can evolve later, with the net in place, instead of being born perfect.

That is why the prudent order fixes the map first and freezes the contracts as early as possible. Contracts between contexts are the most expensive thing to change later, because both sides of every seam hang from them. Map fixed, contracts published, and only then does the elevator descend into the interior of each context being rewritten. That interior is tactical design, and it is the next chapter.

CardDemo • step 6/11

In CardDemo, “account” does not mean the same to the servicing that governs it as to the reports that aggregate it. To servicing it is the live credit line that protects its invariant; to reports it is a row to be summed. The same name, two models:

```
// Servicing context: the account protects its cycle rule
public sealed class Account
{
    public Money AvailableCreditOfCycle() =>
        limit - (cycleCredit - cycleDebit);
}

// Reports context: the account is a row that is aggregated
public sealed record AccountRow(long AccountId, decimal Balance);
```

The target map gives them distinct contexts instead of a single “account” that lies to both.

CardDemo • step 7/11

CardDemo shows the target map in miniature. The core, account servicing and transaction posting, is rewritten, and the account-file hotspot is resolved on the map before it is resolved in code: the account state comes to have a single context that owns its write, and the others consume it by contract. Interest, stable, stays encapsulated behind an open host with its published language. Identity and access, generic, is carved out early and replaced by a hosted service, consumed behind an anticorruption layer. Reports consume downstream through their own anticorruption layer. Three triage moves and three kinds of relationship, one domain, and none of those arrows was dictated.



Figure. CardDemo’s target context map: the hotspot resolved in a single context that owns the account write, with the rest consuming by contract (OHS+PL on the provider side, ACL on the consumer side).

Chapter 13

Tactical design of the core

With the map fixed and the contracts published, the elevator descends one floor and enters each core context. You have a model of the domain, faithful, marked, with the semantics separated from the old structure, and a map that says which contexts get rewritten. Now you decide what each one will be like inside. This is design, in the full sense, and therefore it is the architect's territory, not the agent's. The agent lays the table of options, as we saw. The architect chooses the dish.

The first liberating idea is the one we already sowed in chapter four. **Over a single recovered domain, many architectures fit.** The domain is one. The ways to structure it are many. The map has already grouped the contexts; inside each one you can choose one architectural style or another, the paradigm, procedural, object-oriented or functional, and decide which parts deserve richness of model and which can stay simple. None of those decisions is dictated by the domain. They are all judgements of the architect about what best serves today's constraints, maintainability, performance, team autonomy, whatever matters in your case. This multiplicity is exactly the prize for having separated the semantics from the structure. If you had kept the old structure, there would be no choice. Because you let it go, there is a spread.

The design work splits into two halves of very different natures, and separating them avoids much confusion. There is a mechanical half and a half of choice.

The mechanical half is the translation of patterns of the recovered domain into patterns of the target architecture, when that translation is direct. A domain operation becomes an execution unit of the new architecture. A domain event becomes a message. A batch process becomes a flow. These correspondences are fairly mechanical once the architectural style is decided, and they are just the kind of work the agent does well and fast. Here the concrete frameworks enter as examples. A framework that organizes the code into vertical slices, each around a business operation and not a technical layer, is an example of a target architecture that fits naturally with a domain recovered in terms of operations and rules, because the vertical slice preserves the cohesion of each operation. But the framework is the example. The durable concept is that there is a part of the translation to the new architecture that is mechanical and delegable.

In practice. In the .NET ecosystem, a vertical-slice architecture framework such as Wolverine organizes each business operation into its own slice, with its message and its handler, instead of spreading it across technical layers. The CardDemo operation that authorizes a transaction and updates the balance, today spread across posting programs that share the account file, then translates into a slice with its input message and its handler; a domain event, into a message; the batch interest calculation, into a flow. Other styles, such as the hexagonal, reach the same place by another road. The framework is the instance; what is durable is that the slice follows the business operation, not the technical layer.

The half of choice is where the architect lives. Here are the decisions the domain does not dictate. Do you turn a concept the old system held anaemic, a bag of data without behaviour, into a rich aggregate that protects its own invariants? Do you redraw a boundary the old system had wrong? Do you turn a loose datum into a value object with meaning of its own, like the account balance, today a numeric field in the file, into an amount that always carries its currency and refuses to be added to a different one? Each of these is a design choice, and each has consequences, not only on the quality of the new system but, crucially, on how expensive the later verification will be. An architecture more faithful to the old structure is cheaper to verify, because it resembles the oracle more. A more ambitious architecture, one that redraws boundaries, is

more expensive to verify, because it diverges more. The agent can and should quantify that consequence for each option. The architect decides with that information on the table.

There is a recurring decision that illustrates well why human judgement is irreplaceable here, and it is the relocation of an invariant. In the old system, a rule could be applied in several places at once, repeated, because several parts of the system wrote the same datum and each checked the rule on its own. On redesigning, the natural thing is to relocate that rule to a single place, the aggregate that owns the datum, so that it protects it once and well. Mechanical, in appearance. But there is a trap that only judgement detects. Relocating the rule to a single site assumes that all the old copies of the rule were identical. And if they were not? And if one of the parts that wrote the datum applied the rule and another did not, deliberately, because in that concrete flow the rule was not meant to apply? Then consolidating the copies into one is not a cleanup, it is a change of behaviour, and possibly a grave error. Detecting that divergence demands looking at the recovered domain with a critical eye, and deciding whether the copies were the same rule badly repeated or distinct rules that looked alike. That is judgement. The agent can point out that there are several copies and show you whether they differ. Deciding what that difference means and what to do with it is the architect's, and it is exactly the kind of decision that, if you delegate it, blows up on you months later.

There is a prudent order for all of this, and it connects again with 3X. Do not chase the ambitious model on the first attempt. The strategic layer was already resolved in the previous chapter: the map fixed, the old system wrapped in its anticorruption layer, the contracts published. Inside, start conservative. Each context stays thin, almost a tracing of the behaviour the oracle shows, even if anaemic and inelegant. That design is cheap to verify, because it resembles the oracle, so you close the equivalence fast and you have a new system that demonstrably does the same as the old one. Only then, with the safety net in place, do you tackle the ambitious improvements, enriching the tactical model of each context and, if needed, going back up to the map to redraw a boundary, each step with its own verification.

The domain specification, which we will see in the next part, serves as a gate to both designs without changing a comma. It does not matter whether you verify the conservative or the ambitious one: because it is written in terms of the domain and not of any architecture, it measures both with the same yardstick. That is why you can start conservative, close the equivalence, and enrich afterwards without rewriting the specification, because what it measures is the what, which does not change, not the how, which does. First prove you break nothing with a humble design. Then improve, step by step, never letting go of the net. Optimizing before you have the net is, again, exploring where you should have been stabilizing.

This takes us straight to the pillar that makes everything before it safe instead of reckless. You have recovered a domain and decided a new architecture that diverges from the old one. How do you prove, not assume, prove, that the new system means the same as the old one? That is Part IV, and it is where the method stops being a good intention and becomes a verifiable commitment.

CardDemo • step 8/11

The CardDemo operation that authorizes a transaction is today spread across the posting and the file REWRITE. In the new design it becomes a single vertical slice, a message and its handler:

```
public record AuthorizeTransaction(long AccountId, Money Amount);

public async Task<Result> Handle(AuthorizeTransaction c)
{
    var account = await accounts.Load(c.AccountId);
    return account.Authorize(c.Amount); // applies the cycle rules
}
```

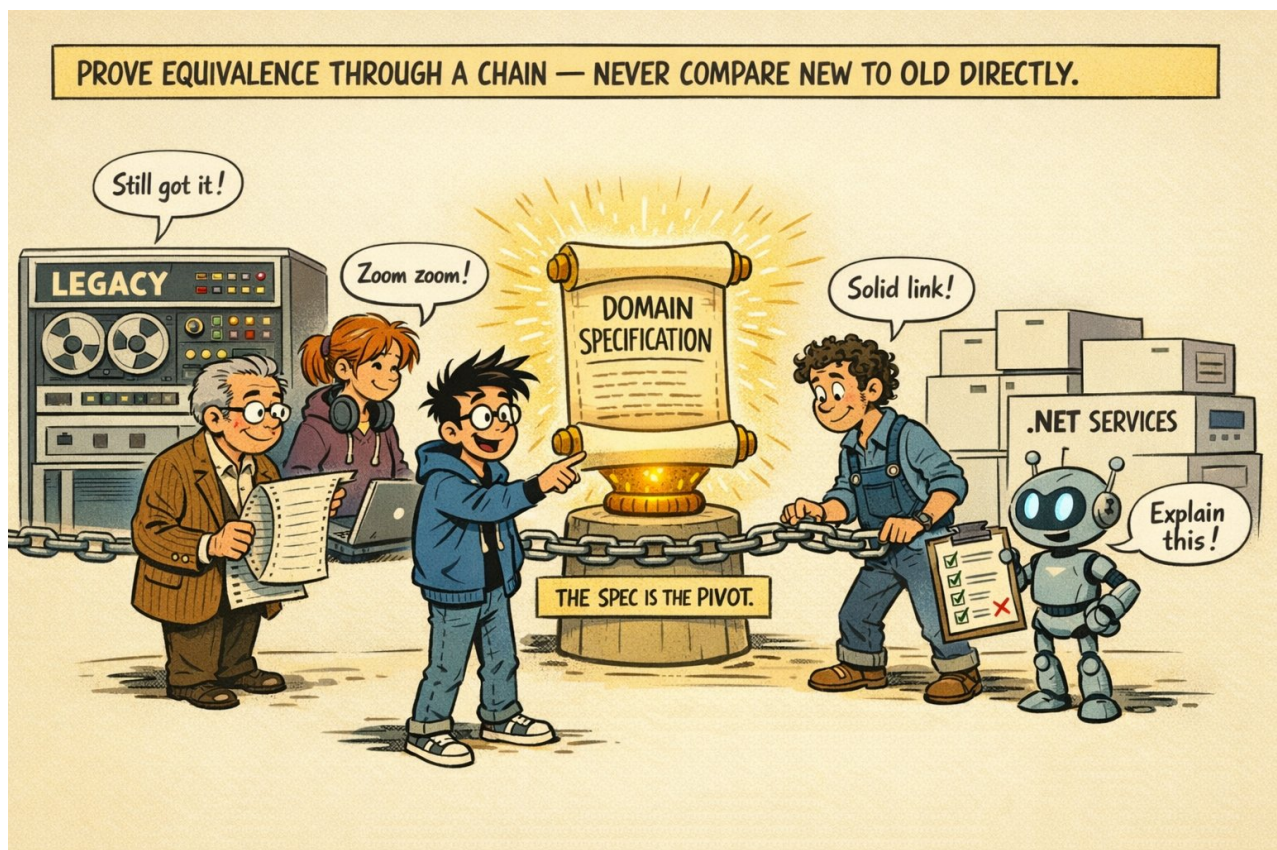
The batch interest calculation, which walks the accounts at the close of the cycle, becomes a separate flow. Each slice follows a business operation, not a technical layer.

Part IV

Proving you did not break it

Chapter 14

The transitive chain



Here is the problem in its crudest form. You have the old system, which works but whose design you abhor. You want a new system, with a design you have chosen and that diverges on purpose from the old one. And you have to prove that the new behaves the same as the old in everything that matters. The temptation is obvious. Compare the new with the old, directly. Same input to both, compare the outputs, if they match, equivalent.

That direct comparison is harder than it looks, and sometimes impossible to do well, for a reason that comes from chapter four. The new system and the old one diverge inside on purpose. They have different boundaries, different data structures, they may even speak different languages at their interfaces, because the old one speaks in terms of its old mechanism and the new one in terms of its clean domain. In CardDemo, the old one answers with a return code and a positional COMMAREA where the new one answers “transaction authorized” with a domain object: they say the same thing in two languages that do not resemble each other. Comparing two systems that do not even present the same shape of input and output demands a translation layer between them, and that translation layer is new code, unverified, that interposes itself

right where you want certainty. You end up trusting your proof of equivalence to a translator whose correctness no one has proven. Direct comparison puts an unknown at the centre of the proof.

The solution is a detour that turns out to be straighter than the direct line. **Never compare the new system against the old one directly. Build a transitive chain of equivalences.** The idea is to introduce intermediate steps chosen so that each link of the chain is easy to verify, and let transitivity do the rest. If A is equivalent to B, and B is equivalent to C, then A is equivalent to C, without ever having compared A with C directly.

Let us see the chain. The first link is to build a replica of the old system that preserves its structure, not its design, its structure. This is exactly the oracle of chapter three, the copy that runs fast and local. Because it preserves the old structure, proving that this replica is equivalent to the original old system is easy, because they are almost identical inside, so the comparison is almost an identity and can be done with a fine verification, comparing even internal details, because the structures correspond. This link is cheap precisely because there is no design divergence to bridge. Once closed, you have a faithful replica of the old behaviour that lives on your machine and answers in milliseconds. And, an important point, from here on the original old system can withdraw from the equation of daily work, because its behaviour already lives, faithfully, in the replica.

The second link does not compare systems, it captures behaviour. You use the faithful replica as oracle and observe, systematically, what it does over a well-chosen set of inputs. The result of that observation you write as an executable specification, a collection of cases that say, given this situation and this request, the observable behaviour is this. That specification is the next link of the chain, and it has a magical property. It is expressed in terms of the domain, not in terms of the structure of any system. It does not say what the old system does inside nor what the new one will do inside. It says what is observed, in domain language. That is why it can serve as a pattern for both.

The third link is the new system. You build it, with its divergent design, and you prove that it satisfies the captured specification. You do not compare it with the old one. You compare it with the specification, which is in domain language and therefore does not demand a dubious translation layer, it only demands that the new system, expressed in its own clean domain, produce the observables the specification dictates.

And there transitivity closes. The old system is equivalent to its faithful replica, link one, fine verification. The faithful replica produces the specification, link two, by direct observation. The new system satisfies the specification, link three. Therefore the new system is equivalent to the old one, without ever having compared them directly, without ever having built that dubious translation layer between two systems that diverge inside. The specification, in domain language, is the pivot that connects the two worlds without touching them to each other.

There is an additional elegance in this chain worth noting. The granularity of the verification follows the structural resemblance of each link, and this is a general principle worth remembering. Where two things resemble each other a lot inside, link one, old against replica, you can and should verify fine, looking at internal details, because the correspondence exists. Where two things diverge inside on purpose, link three, specification against new system, you cannot look at internal details because they do not correspond, so you verify only what is observable from outside, and the observable is named in the language of the domain, in the problem space, not in technical terms of any solution. The granularity of the proof adjusts to how much the things you compare resemble each other. Forcing a fine comparison between things that diverge is the mistake the transitive chain avoids.

The chain also reconciles the triage with the verification, and the image is clean: **the triage decides how much chain you walk.** For the COBOL you decided to encapsulate and relocate, the first link is all the verification it needs: once the fidelity of the recompiled replica is proven, that context is already where it had to be, behind its contract. Only the core you rewrite walks the entire chain, because only it diverges by design. The chain is not a uniform toll; it is a road of which each workload walks the stretch its destination demands.

It remains to develop what exactly that captured specification is, how it is written so that it is at once a test pattern, a regression net and documentation, and why its independence from the language of the two systems is what protects your freedom to redesign. That is the next chapter.

CardDemo • step 9/11

The old CardDemo speaks through a positional COMMAREA, the copybook C0C0M01Y, where each datum lives in its place by offset:

```
01 CARDDEMO-COMMAREA.  
  05 CDEMO-ACCT-ID      PIC 9(11).  
  05 CDEMO-ACCT-STATUS  PIC X(01).
```

The new one speaks through a domain object:

```
public record TransactionAuthorized(long AccountId, Money RemainingCredit);
```

They say the same thing in two languages that do not resemble each other. Comparing them directly would demand a translator between the two, unverified code placed exactly where you want certainty. That is why the chain does not compare them against each other: it connects them through the specification, written in domain language, which serves as the pattern for both.

Chapter 15

Behaviour as executable specification

The captured specification is the pivot of the whole verification, so it deserves a whole chapter on what it is and why it has the shape it has. The central idea is that the observed behaviour of the oracle is written as an executable specification, a collection of cases a machine can run and check, not a prose document someone has to interpret.

The most useful form I know for this is the tabulated example, in the style of given this, when that, then this other. Each case describes a starting state, given this, an action, when that, and the expected observable, then this other. A row of the table is a case. The whole table is the specification of a domain operation. This form has an enormous pedagogical virtue, because it reads almost like natural language and a business expert who does not program can review it, and at the same time it is executable, because each row can be run against a system and check whether the real observable matches the expected one. This idea is not new. Ward Cunningham introduced it in 2002 with FIT, the Framework for Integrated Test: tables a business expert can read and that a machine at the same time executes.

The most important thing about the specification, and what makes it the pivot of the transitive chain, is the language it is written in. **The specification is written in the ubiquitous language of the domain, not in the language of either of the two systems.** The old system speaks in terms of its mechanism, its memory structures, its technical codes. The new system speaks in terms of its clean domain. If you wrote the specification in the language of the old one, you would tie the new system to the old shape and lose the freedom to redesign that cost you so much to win. If you wrote it in the language of the new one, you could not use it to verify against the old one. The only form that serves both is a third language, that of the pure domain, independent of both implementations. That is the language in which the business expert would describe the rule knowing nothing about computers. With a precision chapter twelve left established: the ubiquitous language is one per context, not one for the whole business. Each context is specified in its own language, and the cases that cross contexts speak the published language of the contract that joins them.

This has a concrete architectural consequence. Because the specification is in domain language and each system speaks its own, you need a translation piece per system, that takes a case in domain language and runs it against that concrete system, translating the input into the system's language and the output back into the domain. There is one of these pieces for the old one and another for the new one. The virtue of isolating the translation in these pieces is that the dependency on each system's language is confined there, in a small and well-bounded place, so bounded that it can be unit-tested and tested thoroughly, exhausting its cases, instead of contaminating the specification. The specification stays pure, in the domain, reusable, and the dirt of each concrete language lives isolated in its translation piece. That purity is what protects your freedom to redesign, because you can change the whole new system, its language included, its entire technology stack, and the specification still holds, you only change its translation piece.

The specification captures the complete domain observable, and you have to be careful about what complete means. It means not only the direct answer to the request, but also the side effects that matter to the domain, the change of state the operation leaves behind. If an operation, besides answering, modifies the state of an entity, that change of state is part of the observable and must be in the specification. But, and this is key, those side effects are observed through domain queries, asking the system for its state in domain terms, not spying on its data store from inside. Spying on the store would break the black box and tie the proof again to the concrete storage structure, forbidding redesign. You ask for the state in domain language, just as you asked for the answer. In CardDemo, if authorizing a transaction reduces the available

credit, you do not read the account file: you ask the system for the account balance in domain language and check that it dropped by what it should.

There is an honest limit worth declaring. Complete means all the domain observable that matters, not literally every effect. Purely technical effects, an entry in a log, an internal counter, a plumbing status code, are not captured, because they are not domain, they are mechanism, and comparing them would be comparing precisely what you have the right to change. And there is a deeper limit, you can only capture what the system exposes. If there is a domain effect that no query brings to light, it falls outside the specification, it is a blind spot. Capturing everything observable is the goal, but it demands that the system expose, via domain queries, all its domain effects, and that does not come for free, it has to be designed.

To finish, an observation on why this specification is so profitable. Each row of the table lives three lives at once. While the old system or its replica are still alive, the row is a parity test case, it checks that old and new match. When the old one retires, the same row becomes a regression test of the new system, it checks that no future change breaks the inherited behaviour. And always, from the first day, the row is living documentation of the domain, in language the business understands, and that agents also understand and can render in many forms, in tables, in markdown, in diagrams. It does not lie, because it is executable and would break if it lied. Few things in engineering give three uses for the price of one. The domain specification is one of them, and that is why it is worth the effort of capturing it well.

One last piece is missing for the verification to be solid, and it is the subtlest of all. What exactly you compare when you compare the observable, and in what order you do things. That is the last chapter of this part.

CardDemo • step 10/11

In CardDemo, authorizing a transaction reduces the available credit. To check the effect you do not read the account file: you ask the system for the balance in domain language and check that it dropped by what it should. Reading the store would tie the proof to the storage structure; asking for the domain leaves it free.

```
// breaks the black box: ties the proof to storage
var balance = ReadAccountFile(accountId).BalanceField;

// domain observable: asks in domain language
var balance = await accounts.GetAvailableCredit(accountId);
```

Chapter 16

Black-box parity and the law of order

We already know that we compare what is observable from outside, not the innards. We call that way of verifying black-box parity, because it treats each system as a closed box of which only the inputs and outputs are seen. But black box does not mean comparing blindly everything that comes out. It means choosing carefully which of what comes out counts as a domain observable and which is technical noise to be ignored. That choice, what is compared, is the most delicate dial of the whole verification, and miscalibrating it ruins the proof in one of two opposite directions.

If you compare too much, including in the comparison technical details the new system has the right to do differently, the proof fails constantly over differences that do not matter, a different internal code, a different date format, a different field order. You drown in false positives, the proof becomes noise, and you end up ignoring it, which is the worst thing that can happen to a test. If you compare too little, excluding things that were domain observables, the proof passes even though the behaviour really changed, and it gives you a false security, which is even worse, because it lets you break the system with a clear conscience. Between comparing too much and too little there is a correct calibration, and finding it is the craft.

The calibration rule is this. Classify each field of the observable into one of several categories. The domain results and the resulting domain state are strict comparison, they have to match exactly, down to the last cent of the account balance after closing the cycle, because they are the semantics you promised to preserve. The purely technical fields, mechanism with no domain consequence, are excluded, because they are exactly what you have the right to change. The non-deterministic fields, a generated identifier, a timestamp, are normalized before comparing, reduced to a canonical form that ignores what legitimately varies. And there is a dangerous category, the ambiguous or entangled fields, where a technical value hides a domain consequence, and those have to be disentangled carefully, comparing the domain consequence and not the technical value that carries it. And beware of what looks technical and is not. The order of the fields of the statement that Reporting consumes, or the format of the pricing group that governs the rate lookup, look like mechanism, but if someone downstream depends on them they are domain rules, and they go to strict comparison.

The golden rule of this classification is to compare strict by default and exclude only with evidence. The burden of proof falls on exclusion, not on inclusion. If you doubt whether a field matters, you compare it, because the risk of comparing too much is annoying noise, while the risk of excluding too much is dangerous blindness. You exclude a field only when you can prove it is technical and that it drags no domain consequence. And that proof should be traceable, each excluded field traces to a concrete piece of mechanism with no domain effect. Excluding for convenience, because the field is a nuisance, is opening the door to blindness.

The case of errors illustrates the subtlety well. When an operation fails, do you compare the failure mode or the domain outcome? The right answer is almost always the domain outcome. If the old system rejects an operation by a business rule, and the new one rejects it by the same business rule, they are equivalent, even though inside the mechanism of the rejection is completely different. What matters is that the operation was rejected for that domain reason, not how the rejection was implemented. By contrast, if the old one fails by a technical fault, that is not domain, it is mechanism, and it should not form part of the comparison. Telling the business rejection from the technical failure is exactly the disentangling of chapter four, where we warned that there the silent loss of semantics hides. Well-calibrated parity is where that distinction becomes operational.

The second theme of the chapter is missing, and it is a rule of order that has the rank of a law in this method. **Equivalence is closed before any quality refactor begins.** That is, first you prove that the new system is equivalent to the old one, with its initial conservative design, and only afterwards, with that equivalence proven and green, do you begin to improve the internal quality of the new system, to refactor, to enrich, to polish. Not the other way round. Never the other way round.

The reason is structural, not a preference. Refactoring is changing the structure while preserving the behaviour. To do it safely you need a net that tells you whether you have broken the behaviour while you changed the structure. That net is precisely the equivalence specification. If you refactor before having the equivalence closed, you refactor without a net, and a change that breaks the behaviour goes unnoticed, because you have nothing to detect it. Worse still, if you do both things at once, redesigning for equivalence and refactoring for quality mixed together, when something fails you will not know whether it failed because your redesign was not equivalent or because your refactor broke something. You will have lost the ability to diagnose, which is the same sin as the double jump of chapter two, making two distinct changes in the same move.

This law of order has an authority worth embodying in something, a guardian that enforces it. Call it what you like, what matters is its function. It is the equivalence authority. It owns the specification, which is its evidence. It runs the comparisons. It keeps the green runs, with their coverage and their provenance, so that there is a record of what has been tested and with what force. And it enforces the order, it does not let the quality refactor begin until the equivalence is closed. It is not just a tool that generates tests. It is the arbiter that guarantees the method is followed in the right order, because the history of modernization disasters is full of teams that refactored over sand, over code they believed equivalent without having proven it, and that discovered in production that it was not.

With the whole verification standing, transitive chain, domain specification as pivot, well-calibrated black-box parity, and the law of order guarded, you have the safety net that makes redesign engineering and not a bet. But the longest and least glamorous part of any real modernization is still missing. The old system is not switched off the day the new one is ready. The two coexist, sometimes for years. Coexisting well is an art of its own, and it is Part V.

CardDemo • step 11/11

Two CardDemo fields look like mechanism and are not. The order of the statement fields that Reports consumes, and the format of the pricing group that governs the rate lookup, are read by someone downstream, so they are domain rules and go to strict comparison. The account balance after the close of the cycle is compared to the last cent. And the edge case is written as a specification row in domain language:

```
Given an available credit of 100.00 in the current cycle
When a transaction of 100.01 is authorized
Then the transaction is rejected for exceeding the cycle credit
```


Part V

Coexisting with two systems

Chapter 17

The strangler, honest

Almost everything written about modernization treats the change from old to new as an instant, the day of the change, as if there were a switch. In the reality of any system that matters, there is no switch. There is a period, often long, in which the old system and the new one coexist, serving the business operation between them, while the new one takes on responsibilities little by little. Designing that coexistence is first-class architecture work, not a deployment detail, and underrating it is one of the most common causes of a technically correct modernization ending in operational disaster. And the coexistence is not only between the old and the new core: the estate that comes out of the triage includes encapsulated contexts that do not migrate, they coexist to stay, behind their contract. The strangler governs the part that does move, the core being rewritten.

The classic pattern for this is the strangler, which Martin Fowler named after a fig that grows around a tree until it replaces it. The idea is to put a facade in front of everything, that receives all the requests, and that for each one decides whether the old system or the new one serves it. At first almost everything goes to the old one. Little by little, as each part of the new one is proven equivalent, the facade begins to route that part to the new one. The fig grows. The old tree shrinks. One day the old one no longer serves anything and can be retired.

Described this way it sounds clean, and the routing part really is. The facade that decides whom to send each request to is, conceptually, simple. The problem, and here comes the honesty of the title, is that routing is the easy and visible part, while the hard and invisible part is underneath, in the data. And almost all the literature gets excited about the easy part and goes quiet about the hard one.

Let us see why the data are the problem. While the two systems coexist, there is a business datum, say the state of a central entity, that some operations the new system already serves and others the old one still serves. If an operation that has already migrated writes that datum to the new store, and another that has not migrated writes it to the old store, the datum has split in two. There are two versions, in two stores, diverging. The new system believes one thing about the entity, the old one believes another, and the business is corrupt without anyone noticing until it is too late. The facade that routes operations does not resolve this at all, because the problem is not in the operations, it is in the state they share.

From here comes the principle that governs the slicing, and it is less obvious than it seems. **Cut along the ownership of writes, not where it is convenient for you.** That is, group to migrate together all the operations that write the same datum. If four distinct operations write the state of the central entity, those four migrate at once, in the same step, or none migrates. If you migrate two and leave two in the old one, you split the state and corrupt it. The boundary you cut along has to respect who owns writing each datum, because only that way do you guarantee that at every moment a datum is written by a single system, and therefore cannot diverge.

This explains an order of slicing that would otherwise seem arbitrary. The parts of the system that are peripheral, that read a lot and write little or write data only they touch, are easy to migrate first, because cutting them splits no shared state. The central parts, where several operations fight to write the same critical state, are the ones to migrate last and in a block, because they are entangled by the ownership of writes. The core migrates whole or it does not migrate, and it is left for the end precisely because it is the knot. Whoever tries to migrate the core in little pieces, one operation today and another next month, is splitting the most critical state of the business at every step. It is the shortest road to silent disaster.

The facade, moreover, does double duty if you design it well. The same translation piece that in the verification converted between the domain language and that of each system serves here as a bridge between the two systems during coexistence. It is an example of a transitional architecture, in the sense of Fowler's legacy-displacement catalogue: scaffolding that is installed to hold up the replacement and that is removed as soon as it is no longer needed. It is no coincidence. It is the economics of the method rewarding coherence. A piece well designed to isolate the dependency on a language serves equally to verify and to coexist. We will see it again in the next chapter, where coexistence becomes the last test before the definitive change.

For now, the lesson is sober. The strangler is an excellent pattern, but its famous part, routing, is the easy one. The part that decides whether your modernization lives or dies is the coexistence of data, and it is governed by one rule: cut along the ownership of writes, so that no datum has two owners at once. If you internalize only one thing from this chapter, let it be that.

Chapter 18

Cutover without faith

The moment comes to really change a part of the system, to stop routing it to the old one and route it to the new one definitively. That moment is called cutover, the cut. And the question is, when is it safe to do it. The lazy answer, when the tests pass, is insufficient and dangerous, because the tests passing once in your environment does not tell you how the new system will behave under real traffic, with all its variety and its rare cases your tests perhaps do not cover.

The method proposes a cutover criterion that rests not on faith but on accumulated evidence, and it has several conditions that must hold at once. The first, the equivalence specification is green, the new system matches the oracle on the whole strict domain observable. The second, the coverage of that specification is sufficient, it has exercised the branches, the boundaries, the edge cases, not only the happy path. A specification green with poor coverage is a false calm, because it only proves they match where you looked, and where you did not look the bugs live. The third condition is the decisive one and the one people in a hurry most skip.

The third condition is to run the new system in shadow over real traffic, for a time, and check that it matches the old one. Running in shadow means that the real production traffic goes to both systems at once, but only the answer of the old one counts, which is still in charge. The new one processes the same traffic in parallel, its answer is compared with the old one's, but it is not used, it does not affect the business. It is a rehearsal with a net, the new one acting on reality without consequences. During that period you accumulate evidence that the new one matches the old one not over the cases that occurred to you, but over the real distribution of what actually happens, including the rare cases that neither your specification nor your imagination had covered. The shadow is where the silent loss of semantics that escaped capture surfaces, because the real traffic is the most exhaustive examiner there is, it tests everything, without mercy and without warning.

The fourth condition closes the criterion. Every divergence observed in shadow has been investigated and resolved, or ratified as acceptable. A divergence in shadow is gold, it is the system warning you of a case where new and old do not match. Each one deserves investigation. Almost always it reveals a fault in the new one that you fix. Sometimes it reveals that the divergence is desired, a change of rule you decided on purpose, and then it is ratified and documented as such. What is never done is to ignore it. Cutover with unexplained shadow divergences is cutover with faith, exactly what the method rejects.

There is a historical lesson behind all this caution, and chapter one put names to it. The modernizations that have ended in disaster headlines, systems that massively corrupted data after the change, almost always share the same root. It was not a tool failure. It was a governance failure. Someone asserted equivalence instead of proving it, cut over on that assertion, and reality charged the difference. The cutover-without-faith criterion, green gate plus coverage plus shadow plus resolved divergences, is the direct answer to that class of disaster. You do not cut over on what you believe. You cut over on what you have proven and confirmed against reality.

There is a fifth condition, and the graveyard of chapter one justifies it on its own. The cutover needs a rehearsed way back: a return path to the old system that has been exercised, not just written, with its data and its deadlines clear. The Australian payroll disaster was not only cutting over badly; it was cutting over with no exit. **The cutover is reversible until it stops being so, and that line is crossed knowingly, not discovered.** While the way back exists, a surprise is an

incident; once it stops existing, the same surprise is a disaster. Knowing at which exact point of the plan that ship gets burned, and deciding to cross it on evidence and not on calendar, is part of the criterion.

And one more piece of honesty, which connects with the graveyard's first lesson. This book concentrates on behavioural equivalence, but the cutover also moves state: the initial load of the data, its reconciliation against the source, the counts that tally. That discipline has its own craft and this method does not detail it, but it does demand it at the gate: data reconciliation in green is a condition of the cutover as much as parity in green, because data parity and behavioural parity are two different proofs, and the graveyard is full of those who presented one as if it were the other.

When all the conditions hold, the cut can be made, and it is worth making it gradual even then. Instead of sending all the traffic to the new one at once, you send it a small fraction as the system in charge, you watch, and if it holds, you raise the fraction little by little. It is prudence carried to the last metre. Even with all the accumulated evidence, a gradual change leaves you a way out if something unexpected appears with the new system really in charge.

An honest technical warning so as not to sell the shadow as free. Running in shadow demands that the new system process real traffic without causing real effects in the world. Its writes have to go to a shadow store, not the production one. Its external effects, sending a notification, calling a third party, have to be suppressed, because otherwise the rehearsal becomes reality. For operations of pure calculation and internal state change, this is clean. For operations with visible external effects, it is more delicate, you cannot really send the notification twice, so you suppress the external effect and compare only the internal decision that would have triggered it. Designing that suppression well is part of the cost of the shadow, and it is worth budgeting for it instead of discovering it late.

With the cutover done, part by part, the new system takes over command and the old one is left without work. The last question of coexistence arrives, and it is subtler than it seems. How the old system is retired without losing what only it knew.

Chapter 19

Retiring the oracle without losing the reference

The old system, and its faithful replica that has served as oracle, have been your source of truth throughout the modernization. Every time you doubted what the new system should do, you asked the oracle. Now the new one is in charge and the old one is superfluous. The temptation is to switch it off and celebrate. But switching off the oracle is to lose forever the only source that knew, with certainty, what the old system did in each case, including those you never captured. It has to be retired carefully, because it is a care that cannot be repeated, once switched off, there is no going back.

The sequence rule is blunt. **Capture everything you will need from the oracle before you retire it, because afterwards you will not be able to re-derive it.** While the oracle lives, you can ask it any new question that occurs to you and obtain the truth. When it dies, the only reference that remains is what you captured, the frozen specification. If a case is not in the specification and the oracle no longer exists, that behaviour has been lost as a source of truth, you are left only with what the new system does, which you can no longer check against anything. That is why the oracle's retirement is gated by a condition, the specification captures everything we are going to need, at the coverage we have agreed is sufficient. While that condition does not hold, the oracle is not switched off.

Once everything is captured and the oracle is retired, the frozen specification becomes the permanent reference, and here you see why we insisted so much on its shape. The specification, which was a parity pattern while the oracle lived, becomes the regression net of the new system, now only with its translation piece, without the old one's that no longer exists. Every future change of the new system is checked against that frozen specification, which guarantees that the inherited behaviour is preserved. And if ever an inherited rule has to be changed deliberately, that change is made by editing concrete rows of the specification, explicitly and ratified, not by accident. The specification is the memory of the old system, written in a form that outlives its author.

There is a practical cost nuance worth taking advantage of, and it distinguishes two retirements people confuse. One thing is to retire the old system for real, the original, the one that ran on the expensive platform, and that is expensive and is done at the definitive cutover, freeing the cost that justified the whole modernization. Another thing, very different, is to retire the faithful replica, the local oracle, which runs cheaply on your own infrastructure. The replica is cheap to keep switched on. So a prudent strategy is to retire the original old system as soon as the cutover allows, to capture the saving, but to keep the faithful replica a while longer, as a reserve oracle. If after the cutover a surprise appears in production, a case the new system handles in a doubtful way, you can ask the replica again what the old one would have done, and disambiguate. The replica is the cheap insurance you keep after switching off the expensive thing, and you retire it too when accumulated confidence makes it no longer worthwhile.

It is time to close with the honest residue, because this method does not promise absolute certainty and pretending it does would betray everything said. When the oracle, original and replica, no longer exists, the specification is the only reference, and the specification is finite. Every behaviour of the old system that was not captured, the paths that were never exercised, the rare cases that not even the shadow traffic triggered, has become irrecoverable from its original source. If any of those cases wakes one day in production, there is no longer an oracle to ask what the right thing was,

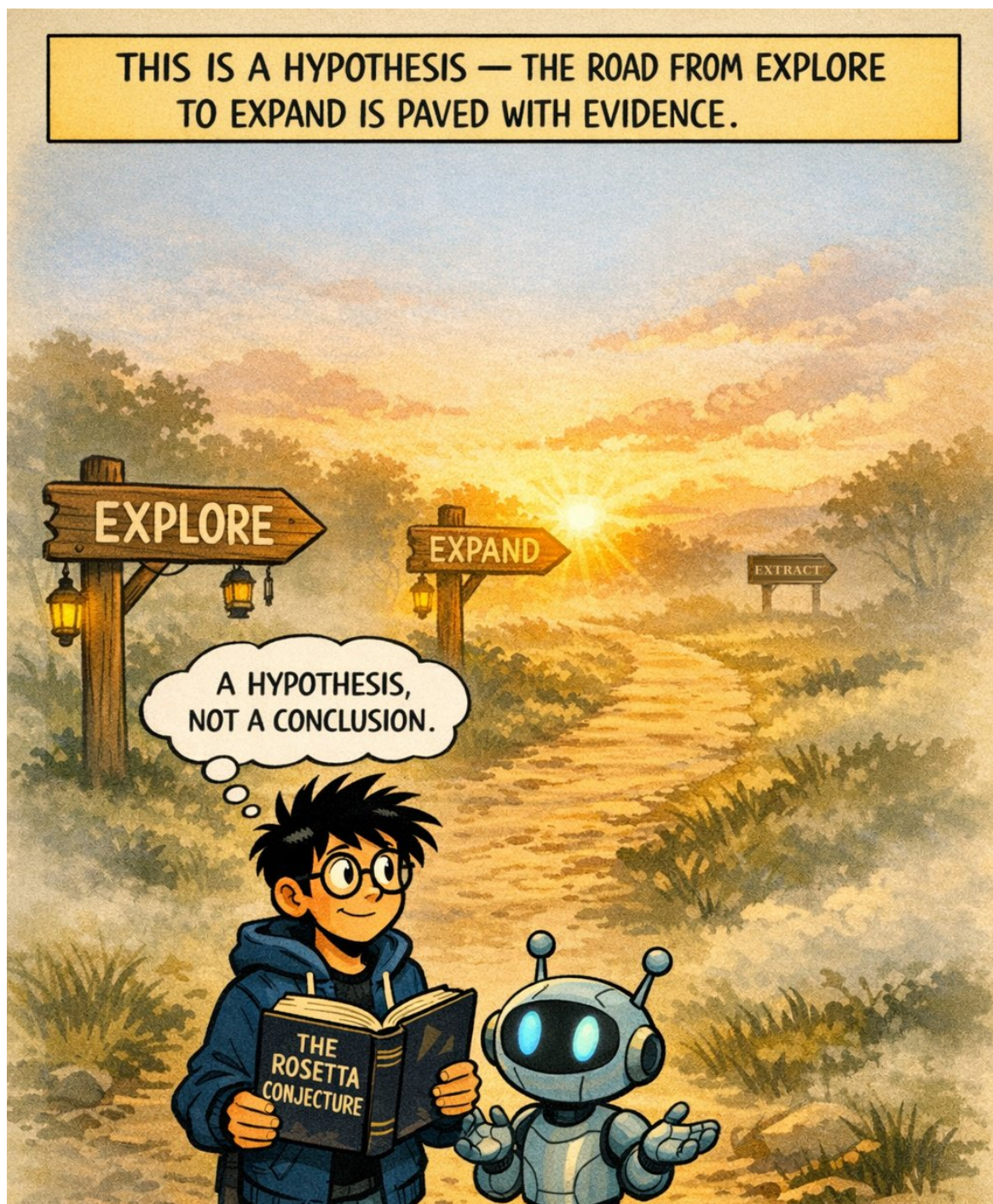
all that remains is to decide it anew, now as design, not as recovery. It comes in through the door of bugs and is treated as one more: it is reproduced, it is decided what it should do, it is specified and covered, like any defect that appears in production. This is not a failure of the method, it is a limit of reality, no process can capture what never manifested. What the method does is bound that residue and make it visible, capture widely while the oracle lives, run the shadow over real traffic to drag the rare cases into the light, and mark honestly the gaps that remain. After retiring the oracle, reality is the only teacher that remains. The method makes sure you reach that point having learned from the oracle everything that could be learned.

Part VI

From Explore to Expand

Chapter 20

This is a hypothesis



Having got here, it is worth taking off a mask that many technical books never take off. What you have just read is not a method proven against hundreds of production migrations. It is a hypothesis, coherent and reasoned, about how modernization ought to be done when you take seriously that it is a domain problem and take advantage of the fact that now there are agents on every floor of the elevator. It is, in Beck's terms, in the Explore phase. And saying so does not weaken the book, it strengthens it, because an argument presented as what it is endures better than a promise presented as what it is not.

It is worth gathering why the method ought to work, because that is the substance of the hypothesis. It ought to work because it attacks the real cause of failure, confusing modernization with translation, instead of a symptom. It ought to work because it rests on a solid source of truth, the observable behaviour of the living system, instead of documentation that lies or code that deceives. It ought to work because it preserves what really matters, the semantics, and frees what really gets in the way, the old structure, instead of confusing the two. It ought to work because it proves equivalence instead of assuming it, and it proves it by a chain of steps each easy to verify. And it ought to work because it treats coexistence and change with the respect they deserve, instead of pretending a switch exists. None of this do I invent. Each piece rests on known shoulders that have survived their own test of time: Eric Evans's Domain-Driven Design, Gregor Hohpe's architect's elevator, Kent Beck's 3X, Martin Fowler's strangler and Michael Feathers's work on how to understand and change legacy systems. Modernizing well a system that matters was always hard. What has changed is that, with an agent on every floor of the elevator, it is at last tractable. The book's bet is that combining them this way, with the domain at the centre and the agent as augmentation on every floor, produces something more than the sum, a process that earns the word modernization.

But a hypothesis is not a conclusion, and it is worth being clear about what would turn it into something more. Moving from Explore to Expand demands evidence that right now does not exist. It demands walking the whole method over real production engagements, several, different from one another, and seeing not only that it works but understanding why it works well enough to predict when it will not. That ability to predict the failure is Beck's signal of maturity, the boundary between a lucky hunch and a method. Until you have it, the honest attitude is to keep exploring, to walk the elevator in fast cycles, learn from each one, discard without regret what reveals itself wrong, and resist the temptation to freeze and productize too soon.

There are open questions the method itself does not resolve and that the Expand phase will have to address. How does domain recovery scale when the system is not one but a constellation of systems entangled with one another? What happens when the recovered domain reveals that the business itself no longer wants to do what the old system did, when modernization uncovers an opportunity to change the rules and not only preserve them? Where is the real limit of what an agent can recover reliably before human judgement has to carry almost everything? How is all of this governed when there is not one architect but many, with different criteria, over the same system? None of these questions has a closed answer here, and pretending it does would be to put the mask back on. They are the work that remains.

That honesty is, in the end, coherent with the method itself. The whole book preaches that it is better to prove than to assume, that provenance matters, that you have to mark what is not known instead of pretending certainty. It would be incoherent for the book to exempt itself from its own discipline and present itself as proven truth when it is reasoned hypothesis. So here is its provenance, marked as the method demands. This is an argument from principles, not a validated field finding. Its strength is in the soundness of the reasoning and in the quality of the principles it combines, not in evidence that does not yet exist.

If the method is good, the walking will prove it, engagement by engagement, cycle by cycle, in the Explore phase where it now lives, until the evidence justifies scaling it. And if in that walking it reveals itself wrong in some part, the right attitude will not be to defend it, it will be to learn from the failure and redo it, because that is exactly what exploring means. This book is not the end of an investigation. It is the best form I have been able to give, today, to a question that is still open. How a system that matters is truly modernized, putting the domain at the centre, with an agent on every floor of the elevator, and without breaking anything along the way. I know of no other attempt to plant itself at that intersection, the domain, the agents and modernization, promising only what can be proven. If a better one appears, it is welcome, because the question matters more than who answers it. The invitation is to walk it, not to believe it.

The domain directs. The agent makes prudence affordable.

Appendix A

Antipatterns

The antipatterns are the mistakes the method fights, gathered here to recognize them fast. Each is the shadow of a principle of the book.

Translation that believes itself modernization. Transpiling the old system into the new language line by line and declaring the work done. It keeps the old design and the interwoven mechanism, now in modern syntax and therefore harder to spot. It is the root antipattern, the one that gives meaning to the whole book.

Speed over an uncertified pipeline. Accelerating with agents a translation chain that never proved behavioural equivalence. Speed applied to a method that does not work does not produce modernization, it produces the next post-mortem sooner.

Recompilation that believes itself the destination. Making the old system run on a new platform without touching its design, and believing that with that you have modernized. It is a legitimate relocation disguised as transformation. The mistake is not to recompile, it is to believe you have finished. And it is worth anticipating something the book will defend later: recompilation has two fully legitimate uses, as instrument, the oracle of chapter three, and as a deliberate destination for what you choose not to redesign, the triage of chapter eleven. What is illegitimate is not recompiling. It is selling it as modernization of the core.

Killing the COBOL too soon. Rewriting or transpiling code that was stable, correct and doing its job, instead of encapsulating it and leaving it alone. It spends budget and scarce experts on what did not ask for it, and trades optionality for risk.

Refactor first, the net later. Beginning to improve the structure of the new system before having proven its equivalence with the old one. Refactoring without a net, with nothing to detect whether you have broken the behaviour. It violates the law of order, and it is the root of the most notorious modernization disasters.

The double jump. Changing the platform and the design in the same move. When something fails, and something fails, you cannot diagnose whether it was the platform change or the design change. You lose the ability to isolate the cause exactly when you most need it.

The choice disguised as a finding. Slipping a design decision into the recovered model as if it were something observed in the old system. It contaminates the portrait of the domain, because it mixes what the system does with what you want it to do. The discipline of provenance exists to detect it.

Ratification that resolves doubts. Turning, in a ratification meeting, a doubtful conjecture into an accepted fact, by consensus and with no new evidence. Ratification must filter what has a firm source, not bless the doubtful with a signature.

The silent loss of semantics. Discarding as technical mechanism something that hid a business rule. The typical case is the handling of a technical error that actually encoded a domain decision. Where mechanism and logic interweave, there this antipattern lurks, and only the oracle disambiguates it.

Parity that compares too much. Including in the equivalence comparison technical details the new system has the right to do differently. It produces a sea of false positives, the test becomes noise, and the team ends up ignoring it, which is the worst thing that can happen to a test.

Parity that compares too little. Excluding from the comparison things that were domain observables. It produces false security, lets real changes of behaviour through, and invites you to break the system with a clear conscience. Worse than the previous one, because silence deceives more than noise.

The slicing that splits the state. Migrating in little pieces a set of operations that write the same datum, leaving some in the old system and others in the new one. The datum splits into two diverging stores. It corrupts the most critical state of the business without warning. It is avoided by cutting along the ownership of writes.

Cutover with faith. Cutting over to the new system because the tests pass, without having run in shadow over real traffic or resolved the divergences. It is asserting equivalence instead of proving it against reality. The history of modernization disasters is written in large part with this antipattern.

Switching off the oracle too soon. Retiring the old system and its replica before having captured everything that was going to be needed. Once switched off, there is no going back, and the uncaptured behaviour is lost as a source of truth forever.

Productizing in Explore. Taking a method or an approach that worked once and freezing it as a corporate standard before understanding it well enough to predict when it will fail. It jumps from Explore to Extract without passing through Expand. A success is not a pattern.

Appendix B

Glossary

Agent. A system able to read a great quantity of material, reason about it and produce results. In this book, an augmentation of the architect present on every floor of the elevator, that finds but does not choose.

Harness. The state machine with guards that organizes the agent's work. Each state has its goal, its clean context and its bounded tool access, and a transition only happens if a deterministic condition holds.

Architect's elevator. Gregor Hohpe's metaphor. The architect goes up and down between the penthouse, where strategy is decided, and the engine room, where the code executes, connecting floors that would otherwise not speak to each other.

Transitive chain. A verification strategy that avoids comparing the new system with the old one directly. It proves easy intermediate equivalences, old with faithful replica, replica with specification, specification with new system, and lets transitivity conclude the final equivalence.

Anti-corruption layer (ACL). A strategic DDD pattern. A layer a consuming context raises to translate any other context's model into its own, so that the foreign shape does not contaminate it. In this book its most frequent use is to protect the new model from the shape of the old system.

Bounded context. A boundary within which a model of the domain and its language are coherent. The seams between contexts are where you can later cut.

Cutover. The cut. The moment when a part of the system stops being served by the old one and is served by the new one definitively.

Domain-Driven Design (DDD). An approach that models software from the business domain and its language, with concepts such as invariants, aggregates, bounded contexts and ubiquitous language.

Double jump. The antipattern of changing platform and design in the same move. Forbidden because it destroys the ability to diagnose failures.

Executable specification. A collection of cases in the form of given this, when that, then this other, written in domain language, that a machine can run and check. It is the pivot of the transitive chain and lives three lives, parity, regression and documentation.

Fidelity. The commitment to preserve the invariants and the semantics of the old system, not its structure. What the system means is sacred, how it achieves it is free.

GraphRAG. The pairing of a semantic search that proposes candidates and a deterministic graph that verifies them, over a projection of the system. The search proposes, the graph disposes.

Invariant. A rule the domain must always satisfy. Part of the problem space, it survives any redesign.

Published language (PL). A strategic DDD pattern. A shared, well-documented language, usually paired with an open host service, that serves as the common medium in which two contexts communicate. It is the language of the contract, not the contract itself.

Ubiquitous language. The shared vocabulary of a context of the domain, in which the specification is expressed, independent of the language of any concrete system. It is one per bounded context, not one global. Its independence is what protects the freedom to redesign.

Context map. A strategic DDD pattern. The chart of an estate's bounded contexts and of the relationships between them. In this method there are two: the recovered as-is map, which is data, and the target map, which is a decision.

Shared kernel (SK). A strategic DDD pattern. A subset of the model that two contexts share and keep in common, with close coordination. It couples, so it is reserved for what truly must be identical in both.

Customer/Supplier. A strategic DDD pattern. A relationship between two contexts in which the downstream is customer and the upstream is supplier, negotiating priorities and commitments.

Conformist. A strategic DDD pattern. The downstream context adopts the upstream model as is, without translating. Cheap and honest when the other model is good; the opposite of raising an anticorruption layer.

Separate ways. A strategic DDD pattern. The decision not to integrate two contexts and to duplicate the little that is needed, when integrating costs more than it is worth.

Mechanism. The plumbing that makes the logic run on a concrete platform. Old solution space. It is what you have the right to change, as against the business logic, which you must preserve.

Recovered model. A faithful portrait of the domain of the old system, in DDD terms, stripped of mechanism, with each statement marked by its provenance. It describes the current state of the domain, not the desired one.

Oracle. The old system, or its faithful replica, as a source of truth about behaviour. You give it an input, it gives you the correct output by definition. It replaces documentation and memory as a source.

Black-box parity. Equivalence verification that compares only what is observable from outside, domain inputs and outputs, never the innards, because the innards diverge on purpose.

Behavioural parity. What verification pursues: that the new system means the same as the old one in what is observable of the domain. It is not the same as data parity, moving the data intact, which can be achieved while the behaviour breaks.

Uncertified pipeline. A transformation chain, today often agentic, that delivers new code without having proven behavioural equivalence against reality. It is the book's critique of the industry, which applies to Rosetta itself until equivalence is demonstrated end to end.

Provenance. A mark that declares where each statement of the recovered model comes from, and therefore how much one can trust it. Extracted, inferred, attested, unproven. What can cite no source is a choice, not a finding, and it leaves the recovered model.

Faithful replica. A copy of the old system that preserves its structure and runs fast and local, made for example by recompiling to a modern platform. It serves as a cheap, fast oracle. Not to be confused with the new system, which diverges on purpose.

Signals. What the system emits about itself while it works, with its provenance, and that lets the guards decide with grounds. It includes coverage, which puts a number on the residue.

Open host service (OHS). A strategic DDD pattern. A public protocol or service through which a context offers its capabilities to many consumers at once, without exposing its inside. It is the way to encapsulate stable COBOL: it is invoked, not inherited.

On the loop. The architect's position relative to the harness. They do not approve each transition, the guards do that, but they watch the machine and decide where, by design, the machine does not decide. As against being inside the loop.

Shadow. A mode of coexistence in which the real traffic goes to both systems at once, but only the answer of the old one counts. The new one processes in parallel without affecting the business, and its answer is compared. It is the last test against reality before the cutover.

Deterministic substrate. The projection of the system derived mechanically, reproducible and with provenance, over which the agent reasons and against which it verifies. It is not invented by the agent.

Triage. A portfolio decision, after recovery, about what to do with each part of the system according to its subdomain type: the generic to a product, the supporting and stable to encapsulation, the core to rewriting.

Strangler. Martin Fowler's pattern. A facade in front of everything routes each request to the old one or the new one, and gradually moves responsibilities to the new one, until the old one can be retired.

3X. Kent Beck's model. Explore, Expand, Extract. Three phases in the life of an idea, each with its mindset. Confusing them is an expensive mistake. This method lives in Explore.